

Phản biện thiết kế hệ thống — Distributed Watermark Tracker

Distributed Watermark Tracker Team

Ngày 3 tháng 6 năm 2026

Mục lục

1	Bảng ký hiệu và định nghĩa (Glossary)	2
2	0. Khung phản biện trước khi vào thiết kế	4
3	1. Thiết kế Strict Watermark	6
3.1	1.1 Trình bày chi tiết thiết kế	6
3.2	1.2 Phản biện: Vì sao thiết kế như vậy	11
4	2. Thiết kế Heuristic Watermark	11
4.1	2.1 Trình bày chi tiết thiết kế	11
4.2	2.2 Phản biện: Vì sao thiết kế như vậy	17
5	3. Thiết kế Node quản lý Partition	18
5.1	3.1 Trình bày chi tiết thiết kế	18
5.2	3.2 Phản biện: Vì sao thiết kế như vậy	21
6	4. Thiết kế Kafka + Bounded Priority Queue tại Node	22
6.1	4.1 Trình bày chi tiết thiết kế	22
6.2	4.2 Phản biện: Vì sao thiết kế như vậy	26
7	5. Thiết kế Tiered Storage	27
7.1	5.1 Trình bày chi tiết thiết kế	27
7.2	5.2 Phản biện: Vì sao thiết kế như vậy	35
8	6. Thiết kế Coordinator & Aggregator	36
8.1	6.1 Trình bày chi tiết thiết kế	36
8.2	6.2 Phản biện: Vì sao thiết kế như vậy	40
9	7. Giao tiếp giữa các Layer	40
9.1	7.1 Trình bày chi tiết thiết kế	40
9.2	7.2 Phản biện: Vì sao thiết kế như vậy	44
10	8. Thiết kế Failback 5 bước	45
10.1	8.1 Trình bày chi tiết thiết kế	45
10.2	8.2 Phản biện: Vì sao thiết kế như vậy	51

Mỗi mục thiết kế được sắp xếp để đọc lần đầu vẫn nắm được ngay: **ý chính của thiết kế** → **bản đồ thiết kế** → **cách chạy từng bước** → **ví dụ vận hành** → **phản biện vì sao chọn như vậy**. Phần bản đồ luôn đi theo cùng thứ tự: thành phần, dữ liệu vào/ra, luồng xử lý, trạng thái cần giữ, giao tiếp và đầu ra. Mã sơ đồ tách riêng trong thư mục **mermaid/**; tài liệu này chỉ trích dẫn. > Render sơ đồ: `mmdc -i mermaid/<tên_file>.mmd -o <tên_file>.svg`, hoặc dán nội dung `.mmd` vào <https://mermaid.live>. **Bài toán.** Gồm luồng log đến bất tuần tự thành cửa sổ 5 giây trên event-time. Quyết định cốt lõi là *chốt cửa sổ khi nào* — một đánh đổi giữa độ đầy đủ dữ liệu và độ trễ. Hai thiết kế watermark là hai lời giải ở hai đầu của đánh đổi này.

1 Bảng ký hiệu và định nghĩa (Glossary)

Dưới đây là bảng giải nghĩa các ký hiệu toán học, các biến trạng thái và thuật ngữ được sử dụng xuyên suốt tài liệu phản biện:

Ký hiệu / Thuật ngữ Ý nghĩa và định nghĩa trong thiết kế

W_{global}

Watermark toàn cục trong chế độ Strict, dùng để chốt cửa sổ trên toàn bộ Worker.

W_{global_h}

Watermark toàn cục trong chế độ Heuristic, dùng để chốt cửa sổ tạm tính (speculative).

$W_{global_h_prev}$

Heuristic Watermark toàn cục ở chu kỳ trước đó.

LW_i

Local Watermark (watermark cục bộ) của partition i , tiến theo mốc punctuation nhận từ nguồn.

W_h

Heuristic Watermark (watermark dự đoán) cục bộ của partition, tính bằng cách lấy max event-time trừ đi độ trễ hiệu dụng.

L_{eff}

Effective Lateness (độ trễ hiệu dụng), ước lượng bằng phân vị p (ví dụ p99) của độ trễ thực tế thông qua DDSketch.

T_{commit}

Mốc thời gian cam kết được mang bởi Punctuation Token từ Ingestor, chứng minh không còn dữ liệu cũ hơn.

T_{event}

Thời điểm xảy ra sự kiện của bản ghi log (event-time).

$arrival_time$

Thời điểm bản ghi được nhận vào hệ thống (thời gian thực tế của log).

$window_end$

Thời điểm kết thúc của sổ thời gian (window-end).

$window_start$

Thời điểm bắt đầu của sổ thời gian (window-start).

max_event_time

Thời điểm sự kiện lớn nhất đã quan sát được trong partition.

W_{h_prev}

Heuristic Watermark cục bộ ở chu kỳ trước đó.

W_{h_raw}

Heuristic Watermark cục bộ thô trước khi áp dụng rate limit hoặc hysteresis.

$W_{candidate}$

Watermark ứng viên đề xuất trước khi chốt đơn điệu.

now

Thời điểm xử lý sự kiện tại Worker Node (thời gian hệ thống).

p

Phân vị cấu hình để tính độ trễ hiệu dụng (ví dụ $p = 0.99$ cho p99 $lateness$).

e

Event hiện tại hoặc bản ghi log đang được xử lý.

$offset$

Vị trí $offset$ của bản ghi trong phân vùng Kafka.

$term$

Chỉ số epoch/fencing $term$ hiện tại của Coordinator, dùng để chống các lệnh reassign trùng lặp hoặc cũ.

$delta$

Chênh lệch giá trị tích lũy do late event tạo ra trong correction.

$window_id$

Định danh duy nhất của cửa sổ thời gian (thường ghép từ $partition_id$, $window_start$, $window_end$ và mode).

$partition_id$

Định danh phân vùng dữ liệu Kafka.

$worker_id$

Định danh tiến trình xử lý (Worker Node).

$emit_id$

Khóa chống trùng (idempotency key) cho một lần phát kết quả của sổ.

2 0. Khung phản biện trước khi vào thiết kế

Trước khi trình bày từng thành phần, cần cố định bốn điểm để hội đồng hiểu đúng phạm vi thiết kế. **Bài toán đầu vào.** Log không đến theo thứ tự event-time. Một bản ghi có T_{event} nhỏ vẫn có thể đến sau nhiều bản ghi mới hơn. Nếu chốt cửa sổ quá sớm thì mất dữ liệu; nếu chờ quá lâu thì latency tăng. Vì vậy hệ thống không chỉ là "đọc Kafka rồi group-by window", mà là bài toán quyết định thời điểm đóng cửa sổ dưới dữ liệu trễ. **Ràng buộc thiết kế.**

Ràng buộc	Ý nghĩa khi phản biện
Event-time window	Cửa sổ được tính theo thời điểm sự kiện, không theo thời điểm hệ thống nhận được log.
Out-of-order arrival	Cần watermark để biết khi nào đủ an toàn để chốt.
12 Kafka partition / 4 worker	Đơn vị đúng đắn nhỏ nhất là partition, không phải node.
Có crash/failover	State, $offset$ và quyền sở hữu partition phải phục hồi được.
Hai mục tiêu khác nhau	Strict tối ưu correctness; Heuristic tối ưu latency và sửa sai bất đồng bộ.

Tiêu chí đánh giá.

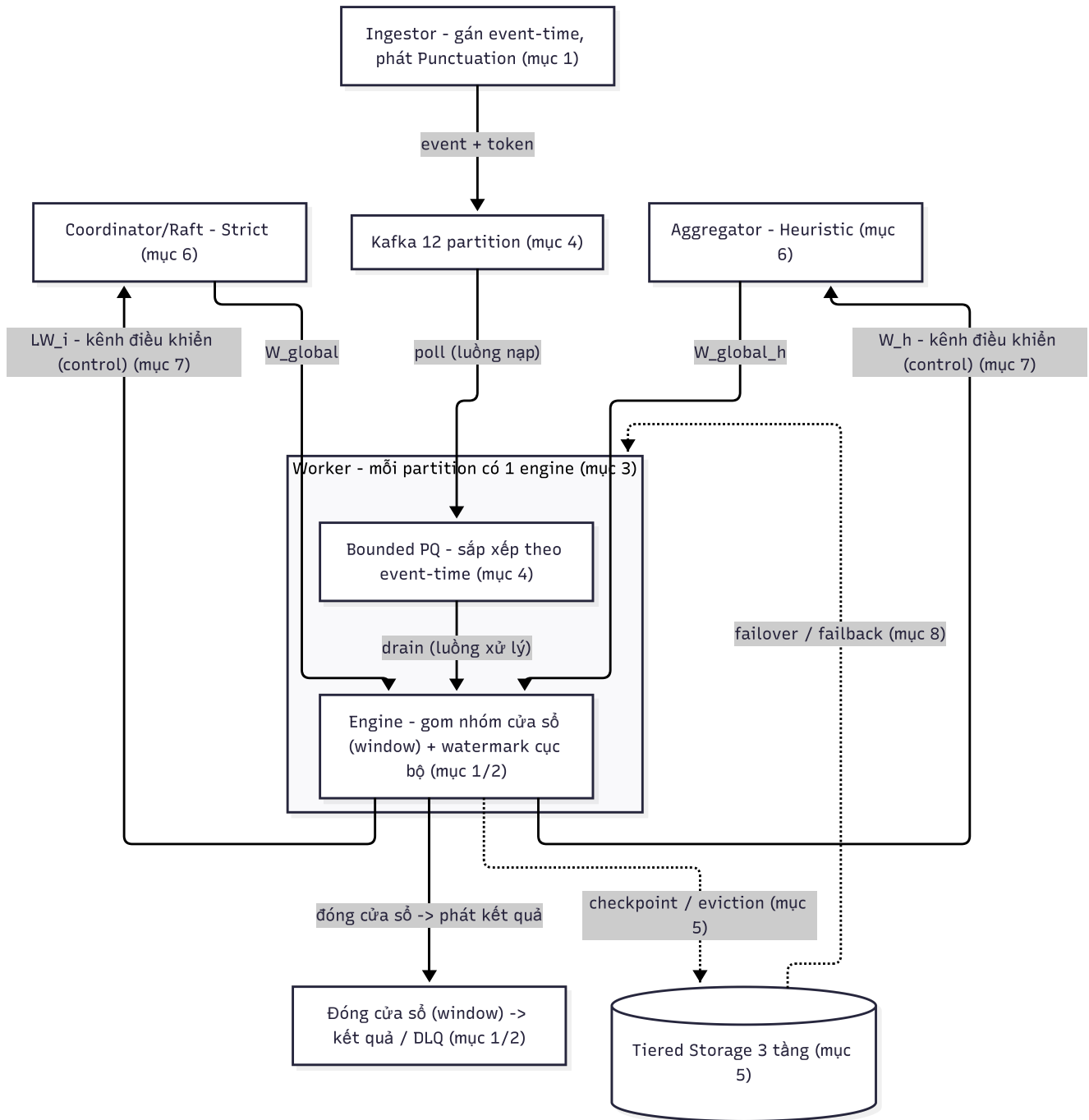
Tiêu chí	Strict Watermark	Heuristic Watermark
Cam kết chính	Không mất dữ liệu trong đường xử lý chính	Latency thấp, chấp nhận late event tức thời
Cách chốt cửa sổ	Dựa trên Punctuation Token và $W_{global} = \min(LW_i)$	Dựa trên $L_{eff} = DDSketch.quantile(p)$ và W_h
Dữ liệu đến muộn	Không được để xảy ra trong cửa sổ đã chốt nếu token đúng	Đưa vào DLQ và phát correction
Control plane	Coordinator HA/Raft vì có reassign/failback	Aggregator Active-Standby vì chỉ tổng hợp watermark
Khi nên dừng	Audit, billing, compliance, cần đúng ngay	Dashboard, phân tích gần real-time, cần latency thấp

Luận điểm xuyên suốt. Mỗi phần "phức tạp" trong thiết kế phải trả lời được một câu hỏi rất cụ thể: nó bảo vệ cam kết nào? Nếu bỏ nó đi thì hỏng ở thuộc tính nào? Các mục dưới đây trình bày theo đúng hướng đó. **Cách đọc từng mục thiết kế.**

Phần trong mỗi mục	Người đọc cần nắm được
Ý chính	Mục này đang thiết kế thành phần nào và cam kết chính là gì.
Bản đồ thiết kế	Trình bày tách riêng từng thành phần, từng loại dữ liệu, từng state, từng kênh giao tiếp và từng output; không gộp nhiều ý vào một dòng.
Cách chạy từng bước	Event/control message đi qua hệ thống theo thứ tự nào, khi nào cập nhật state, khi nào emit kết quả.
Ví dụ vận hành	Một tình huống số cụ thể để kiểm tra người đọc đã hiểu đúng cơ chế.
Phản biện	Nêu rõ phản đề, vì sao phản đề nghe hợp lý, lý do bác bỏ, và kết luận thiết kế phải giữ thuộc tính nào.

Bức tranh tổng thể: 8 mục liên hệ với nhau thế nào. Tám mục dưới đây không phải tám hệ thống rời rạc, mà là các chặng trên cùng một đường đi của một event:

1. Ingestor gắn event-time và (Strict) phát Punctuation, đẩy vào **Kafka** (6).
2. Worker kéo event từ Kafka vào **Bounded PQ** để sắp theo event-time (6), rồi đưa vào **engine của đúng partition** (5).
3. Engine gom window và sinh **watermark cục bộ**: LW_i từ token (3) hoặc W_h từ DDSketch (4).
4. Watermark cục bộ đi qua **kênh control out-of-band** (9) lên **Coordinator** (Strict) hoặc **Aggregator** (Heuristic) (8) để hợp nhất thành watermark toàn cục.
5. Worker nhận watermark toàn cục, **đóng window** và phát kết quả; event đến muộn (Heuristic) đi vào DLQ (3/4).
6. Trạng thái được bền hóa qua **Tiered Storage** (7); khi node chết, partition được bàn giao bằng **Failback** (10) dựa trên checkpoint Tier-2 và fencing token của Coordinator (8).



Hình 1: Sơ đồ xâu chuỗi toàn hệ thống

Cuối mỗi mục có dòng **Liên hệ** chỉ rõ mục đó nối với những mục nào, để khi đọc không tách rời khỏi tổng thể.

3 1. Thiết kế Strict Watermark

3.1 1.1 Trình bày chi tiết thiết kế

Ý chính. Strict Watermark là thiết kế ưu tiên tính đúng ngay tại thời điểm đóng cửa sổ. Watermark toàn cục chỉ được phép tiến khi *mọi* partition đang hoạt động đã vượt

qua cùng một mốc event-time. Nguồn tiến độ không phải là suy đoán từ Worker, mà là Punctuation Token do Ingestor phát ra. Coordinator hợp nhất local watermark theo công thức $W_{global} = \min(LW_i)$. Worker chỉ chốt cửa sổ khi $window_end \leq W_{global}$. Cam kết của mode này là không mất dữ liệu trong đường xử lý chính và bảo vệ exactly-once ở mức window output. **Bản đồ thiết kế. Thành phần và trách nhiệm.**

- **Ingestor:** đọc log đầu vào, gán log vào đúng Kafka partition, phát event và phát Punctuation Token cho từng partition.
- **Kafka topic events:** giữ event và token theo thứ tự $offset$ trong từng partition để Worker có thể replay.
- **StrictWorker:** poll event/token theo partition, chuyển event vào engine đúng partition và gửi heartbeat lên Coordinator.
- **StrictWatermarkEngine:** giữ open window, cập nhật aggregate, nhận token để tăng local watermark LW_i , đóng window khi đủ W_{global} .
- **StrictCoordinator:** nhận heartbeat theo partition, tính $W_{global} = \min(LW_i)$, giữ watermark toàn cục đơn điệu.
- **OutputManager:** phát WindowResult sang results/audit và chống phát trùng theo $window_id$.

Dữ liệu vào.

- **Log/event:** bản ghi nghiệp vụ có event_time, partition key và payload.
- **Punctuation Token:** mốc T_{commit} chứng minh một partition đã phát hết event có $T_{event} \leq T_{commit}$.
- **Heartbeat từ Worker:** map $partition_id \rightarrow LW_i$ kèm trạng thái partition.

Dữ liệu ra.

- **Local watermark LW_i :** watermark cục bộ của từng partition.
- **Global watermark W_{global} :** watermark toàn cục do Coordinator tính.
- **WindowResult:** kết quả cửa sổ đã đóng, có $window_id$ để chống trùng.

Trạng thái cần giữ.

- **Open window:** aggregate tạm thời của các cửa sổ chưa đóng.
- **Closed window:** cửa sổ đã đóng nhưng còn cần emit/checkpoint/purge.

- **Dedup event id:** tránh xử lý trùng khi replay.
- **Kafka** `offset` : vị trí đã xử lý an toàn của từng partition.
- **Local/global watermark:** mốc tiến độ cục bộ và toàn cục.
- **Fencing** `term` : mốc chống lệnh cũ khi control plane thay leader.

Giao tiếp.

- **Ingestor -> Kafka:** gửi event và Punctuation Token trong cùng luồng partition để giữ thứ tự.
- **Worker -> Coordinator:** gửi heartbeat gRPC mỗi 1s, không gộp watermark theo node.
- **Worker -> Coordinator:** kéo `Wglobal` khoảng mỗi 500ms.
- **Worker -> Output:** phát window result sau khi `window_end ≤ Wglobal`.

Đầu ra.

- **Kết quả chính:** window result không speculative.
- **Kết quả phụ trợ:** audit/result metadata để kiểm tra output đã emit và phục hồi không phát trùng.

Thiết kế Output exactly-once và idempotency. Mục tiêu.

- **Không phát trùng kết quả:** cùng một window không được emit hai lần thành hai bản ghi độc lập.
- **Không mất kết quả đã đóng:** window đã đủ điều kiện đóng phải có đường ghi output/audit bền vững.
- **Replay không làm sai output:** khi Worker replay Kafka sau recovery, output đã emit phải được nhận diện là đã tồn tại.

Định danh output.

- `window_id` : định danh chính của kết quả window, thường ghép từ `partition_id`, `window_start`, `window_end` và mode xử lý.
- `emit_id` : idempotency key cho một lần phát kết quả, có thể dùng lại `window_id` nếu mỗi window chỉ có một kết quả chính.
- `offset_range` : khoảng Kafka `offset` đã góp vào window, giúp audit và kiểm tra replay.

- `watermark_at_close`: watermark tại thời điểm đóng window, giúp giải thích vì sao window được phép emit.

State chống phát trùng.

- **Emit registry:** bảng ghi `window_id` nào đã emit thành công.
- **Output status:** trạng thái PENDING, EMITTING, EMITTED, ACKED hoặc FAILED.
- **Audit record:** metadata của output đã phát, dùng để đối chiếu sau recovery.
- **Retry counter:** số lần thử lại khi output sink lỗi.

Luồng emit output.

1. Engine chỉ tạo WindowResult khi $window_end \leq W_{global}$.
2. Worker kiểm tra `window_id` trong emit registry.
3. Nếu `window_id` chưa tồn tại, Worker ghi trạng thái PENDING hoặc EMITTING.
4. OutputManager phát kết quả sang results/audit.
5. Khi sink ack thành công, Worker đánh dấu `window_id` là EMITTED hoặc ACKED.
6. Nếu Worker crash trước ack, recovery đọc emit registry/audit để quyết định retry idempotent thay vì tạo output mới không kiểm soát.

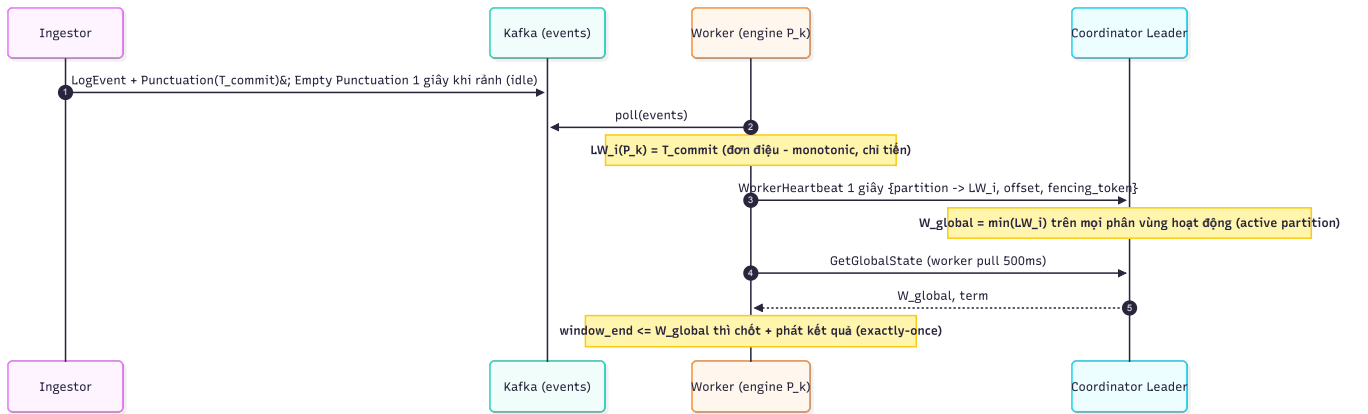
Cách chạy từng bước.

1. Ingestor đọc log đầu vào, gán từng log vào đúng Kafka partition, rồi phát event theo thứ tự `offset` của partition đó.
2. Đến một mốc an toàn T_{commit} , Ingestor phát Punctuation Token cho partition tương ứng. Token này có nghĩa là: với partition đó, toàn bộ event có $T_{event} \leq T_{commit}$ đã được đưa vào Kafka trước token.
3. Worker poll event theo partition. Với mỗi event, StrictWatermarkEngine xác định cửa sổ 5 giây chứa event, cập nhật aggregate của cửa sổ đó, ghi state nóng và ghi nhận `offset` đã xử lý.
4. Khi Worker đọc được Punctuation Token, engine cập nhật $LW_i = \max(LW_i, T_{commit})$ cho partition đó. Worker không tự đóng toàn cục ngay, mà gửi local watermark này lên Coordinator trong heartbeat.
5. Coordinator nhận heartbeat của nhiều Worker, gom watermark theo từng partition và tính $W_{global} = \min(LW_i)$ trên các partition hợp lệ. Watermark toàn cục luôn đơn điệu, nghĩa là nếu lần trước đã là 120 thì lần sau không được tụt xuống 118.

6. Worker kéo W_{global} từ Coordinator. Mọi window có $window_end \leq W_{global}$ được seal, emit kết quả, checkpoint $offset$ /state, rồi mới được dọn khỏi hot state.

Ví dụ vận hành.

- **Tình huống ban đầu:** Hệ thống gồm 3 partition P0, P1, P2.
- **Bước 1 (Heartbeat gửi về):** Worker xử lý P0 báo $LW_i = 120$. Worker xử lý P1 báo $LW_i = 125$. Worker xử lý P2 báo $LW_i = 118$.
- **Bước 2 (Coordinator tính toán):** $W_{global} = \min(120, 125, 118) = 118$.
- **Bước 3 (Worker đóng window):** Cửa sổ $[110, 115)$ có $window_end = 115 \leq 118$ được đóng và emit kết quả. Cửa sổ $[115, 120)$ có $window_end = 120 > 118$ bắt buộc phải giữ lại ở trạng thái mở (chưa được đóng) vì partition P2 chưa an toàn vượt mốc 120.
- **Bước 4 (Cập nhật tiếp theo):** Ingestor gửi Punctuation Token $T_{commit} = 126$ trên P2. LW_i của P2 tăng từ 118 lên 126.
- **Bước 5 (Coordinator cập nhật):** W_{global} mới $= \min(120, 125, 126) = 120$. Lúc này, cửa sổ $[115, 120)$ có $window_end = 120 \leq 120$ đã đủ điều kiện đóng an toàn và emit kết quả.



Hình 2: Sơ đồ

Liên hệ. LW_i bắt nguồn từ Punctuation đi *trong* luồng Kafka (6); heartbeat mang LW_i lên Coordinator (8) qua kênh control out-of-band (9); kết quả và state được bền hóa qua Tiered Storage (7). So với Heuristic (4), Strict dùng chung mọi chặng — chỉ khác nguồn watermark và control plane.

3.2 1.2 Phản biện: Vì sao thiết kế như vậy

Mục Strict phải bảo vệ cam kết "đóng window rồi thì không còn event cũ nào bị bỏ sót". Vì vậy các phương án làm watermark chạy nhanh hơn đều phải bị kiểm tra bằng câu hỏi: có còn chứng minh được dữ liệu đã đủ chưa? **Phản đề 1: Không cần Punctuation Token, Worker tự đoán watermark từ event mới nhất.**

- **Vì sao nghe hợp lý:** bỏ được cơ chế token, Ingestor đơn giản hơn, watermark có thể tiến nhanh hơn vì Worker không phải chờ mốc xác nhận từ nguồn.
- **Lý do bác bỏ:** Worker chỉ nhìn thấy dữ liệu cục bộ của partition mình đang xử lý. Nếu Worker thấy partition P0 đã đến event-time 200 nhưng partition P7 vẫn còn event-time 150 chưa đến, Worker không có bằng chứng để chốt window toàn cục. Tự đoán watermark biến Strict thành Heuristic trá hình, vì hệ thống đang suy luận từ quan sát cục bộ thay vì nhận cam kết đầy đủ từ nguồn.
- **Kết luận phản biện:** Strict bắt buộc cần Punctuation Token. Token là bằng chứng từ Ingestor rằng partition đó đã phát hết event có $T_{event} \leq T_{commit}$; nhờ vậy window đóng không bỏ sót dữ liệu.

Phản đề 2: Partition idle nên bị loại khỏi min() để watermark chạy nhanh hơn.

- **Vì sao nghe hợp lý:** một partition lâu không có log sẽ kéo W_{global} đứng yên; loại nó khỏi min() giúp giảm latency.
- **Lý do bác bỏ:** idle không đồng nghĩa với "đã hết dữ liệu cũ". Nếu partition đó gửi log trở lại với event-time nhỏ hơn watermark đã tiến, các log này sẽ bị coi là late dù thực chất chưa từng được xử lý. Khi đó Strict mất cam kết 0% loss.
- **Kết luận phản biện:** Strict không được âm thầm bỏ partition idle. Cách đúng là Empty Punctuation: Ingestor phát token rỗng cho partition rảnh để chứng minh partition đó đã an toàn đến mốc mới.

4 2. Thiết kế Heuristic Watermark

4.1 2.1 Trình bày chi tiết thiết kế

Ý chính. Heuristic Watermark là thiết kế ưu tiên latency thấp khi upstream không phát được Punctuation Token. Worker không chờ bằng chứng chắc chắn từ nguồn; thay vào đó, mỗi Worker đo độ trễ thực tế của event trong partition mình, đưa mẫu vào DDSketch, lấy $L_{eff} = \text{quantile}(0.99)$, rồi đặt $W_h = \max_event_time - L_{eff}$. Aggregator hợp nhất watermark heuristic bằng $W_{global_h} = \min(W_h)$. Bản ghi đến sau khi window đã đóng không bị bỏ thềm, mà được đưa vào DLQ và phát correction sau. **Bản đồ thiết kế. Thành phần và trách nhiệm.**

- **HeuristicWorker:** nhận event, đo $lateness$, cập nhật engine và gửi watermark heuristic.

- **HeuristicWatermarkEngine:** giữ window state, tính W_h , đóng window sớm theo watermark heuristic.
- **DDSketch/Sliding DDSketch:** lưu phân phối $lateness$ theo partition và trả về quantile như p99.
- **Aggregator:** nhận W_h từ nhiều partition và tính $W_{global_h} = \min(W_h)$ trên partition active.
- **DLQPipeline:** nhận event đến muộn sau khi window đã đóng.
- **CorrectionProtocol:** đọc late event từ DLQ và phát correction cho window result.

Dữ liệu vào.

- **Event:** bản ghi có `event_time`, arrival/processing time và payload.
- **Lateness sample:** giá trị $arrival_time - event_time$ đưa vào DDSketch.
- **Cấu hình quantile:** ví dụ p99 để lấy L_{eff} .

Dữ liệu ra.

- L_{eff} : độ trễ hiệu dụng lấy từ DDSketch.
- W_h : watermark heuristic cục bộ của partition.
- W_{global_h} : watermark heuristic toàn cục từ Aggregator.
- **Late event:** event thuộc window đã đóng, đi vào DLQ.
- **Correction message:** bản sửa kết quả sau khi xử lý late event.

Trạng thái cần giữ.

- **Sketch buckets:** phân phối $lateness$ hiện tại.
- **Open/closed window:** state của window chưa đóng và đã đóng.
- **Late event backlog:** hàng đợi late event chờ correction.
- **Cold-start state:** trạng thái trước khi $sketch$ có đủ mẫu tin cậy.
- **Correction id đã xử lý:** tránh phát correction trùng.

Giao tiếp.

- **Worker -> Aggregator:** gửi W_h qua gRPC chu kỳ ngắn.

- **Worker -> Aggregator/endpoint nhẹ:** kéo W_{global_h} .
- **Worker -> DLQ:** gửi late event khi event thuộc window đã đóng.
- **Correction pipeline -> output:** phát correction message cho window result.

Đầu ra.

- **Kết quả ban đầu:** window result speculative, ưu tiên latency thấp.
- **Kết quả sửa sau:** correction để eventual completeness đạt 100%.

Thiết kế DLQ và Correction Pipeline. Mục tiêu.

- **Không drop late event thầm lặng:** event đến sau khi window đã đóng phải được ghi nhận.
- **Giữ kết quả gần real-time:** window vẫn được phép emit sớm theo heuristic watermark.
- **Đảm bảo đầy đủ sau cùng:** late event được xử lý thành correction để kết quả cuối cùng hội tụ.

Khi nào event vào DLQ.

- **Window đã đóng:** event có `event_time` thuộc window đã emit speculative.
- **Watermark đã vượt window:** `event_time` nhỏ hơn hoặc bằng vùng đã được heuristic watermark cho đóng.
- **Không còn cập nhật trực tiếp:** engine không mở lại window nóng để tránh phá tính đơn điệu của output ban đầu.

Payload DLQ.

- **Event gốc:** payload nghiệp vụ để tính lại aggregate.
- $window_id$: window bị ảnh hưởng bởi late event.
- $partition_id$: partition phát sinh late event.
- `event_time`: thời điểm sự kiện để xác định cửa sổ.
- $arrival_time$: thời điểm hệ thống nhận late event.
- $original_result_version$: version kết quả đã emit trước đó nếu có.
- `late_reason`: lý do đưa vào DLQ, ví dụ `ARRIVED_AFTER_CLOSE`.

Correction state.

- **Late backlog:** danh sách late event chưa xử lý.
- **Correction id:** idempotency key cho bản sửa, thường ghép từ `window_id` và batch late event.
- **Correction version:** version tăng dần của kết quả window sau mỗi lần sửa.
- **Processed correction ids:** tập id đã xử lý để tránh phát correction trùng.

Luồng correction.

1. Worker phát hiện event thuộc window đã đóng và ghi event vào DLQ.
2. Correction pipeline đọc DLQ theo batch hoặc theo window.
3. Pipeline tính `delta` mà late event tạo ra cho aggregate cũ.
4. Pipeline tạo correction message có `window_id`, `delta`, `correction_id` và `correction_version`.
5. Output sink áp dụng correction theo idempotency key.
6. Khi correction ack thành công, pipeline đánh dấu late event/correction là đã xử lý.

Thuật toán Heuristic Watermark. Input của thuật toán.

- **Event hiện tại `e`:** gồm `event_time`, `arrival_time`, `partition_id` và payload.
- **Thời điểm xử lý `now`:** thời điểm Worker nhận/xử lý event.
- **Quantile cấu hình `p`:** ví dụ `p = 0.99` để lấy p99 `lateness`.
- **Window size:** 5 giây.
- **Trạng thái partition:** active, idle hoặc cold-start.

Output của thuật toán.

- **Local heuristic watermark `Wh` [partition]:** watermark cục bộ của partition.
- **Global heuristic watermark `Wglobal_h`:** watermark toàn cục do Aggregator tính.
- **Window result speculative:** kết quả window được emit sớm.
- **Late event hoặc correction:** sinh ra khi event đến sau lúc window đã đóng.

Biến nội bộ tại Worker.

- `max_event_time`: event-time lớn nhất Worker đã thấy trong partition.

- **lateness**: độ trễ của event, tính bằng $now - event_time$.
- **sketch**: DDSketch lưu phân phối **lateness** của partition.
- **L_{eff}**: **lateness** hiệu dụng lấy từ `sketch.quantile(p)`.
- **W_{h_prev}**: watermark heuristic cục bộ trước đó.
- **W_{h_raw}**: watermark tạm trước khi áp dụng hysteresis/rate limit.
- **closed_windows**: tập window đã đóng để phát hiện late event.

Thuật toán tại Worker cho mỗi event (mô tả từng bước).

1. **Đo **lateness****: Worker không đo bằng **offset** Kafka, mà đo bằng chênh lệch giữa thời điểm xử lý và **event_time**.
2. **Cập nhật DDSketch**: mỗi **lateness** sample làm phân phối **lateness** của partition chính xác hơn.
3. **Cập nhật **max_event_time****: Worker nhớ mốc event-time mới nhất đã quan sát để biết dòng dữ liệu đang tiến tới đâu.
4. **Tính **L_{eff}****: lấy quantile như p99 để chứa một khoảng an toàn cho phần lớn event đến muộn.
5. **Tạo watermark thô**: $W_{h_raw} = max_event_time - L_{eff}$. Nghĩa là: chỉ đóng đến trước mốc mà thuật toán tin rằng đã số event cũ đã đến.
6. **Chặn watermark tăng quá nhanh**: rate limit tránh watermark nhảy xa khi vài event mới đẩy **max_event_time** lên đột ngột.
7. **Chống dao động nhỏ**: hysteresis tránh việc watermark thay đổi vì nhiễu nhỏ của phân phối **lateness**.
8. **Gửi lên Aggregator**: Worker không tự quyết định toàn cục; nó chỉ gửi watermark cục bộ của partition.

Thuật toán tại Aggregator (mô tả từng bước).

1. **Nhận watermark cục bộ**: mỗi Worker gửi **W_h** theo partition.
2. **Lọc partition idle**: Heuristic loại partition thật sự idle để một partition rảnh không treo `min()` mãi.

3. **Lấy min trên partition active:** $\min()$ vẫn cần thiết vì window toàn cục chỉ nên đóng đến mốc an toàn nhất trong các partition đang có dữ liệu.
4. **Giữ đơn điệu:** $W_{global_h} = \max(W_{global_h_prev}, W_{candidate})$ để watermark toàn cục không tụt lùi.
5. **Trả watermark cho Worker:** Worker dùng W_{global_h} để quyết định window nào được đóng.

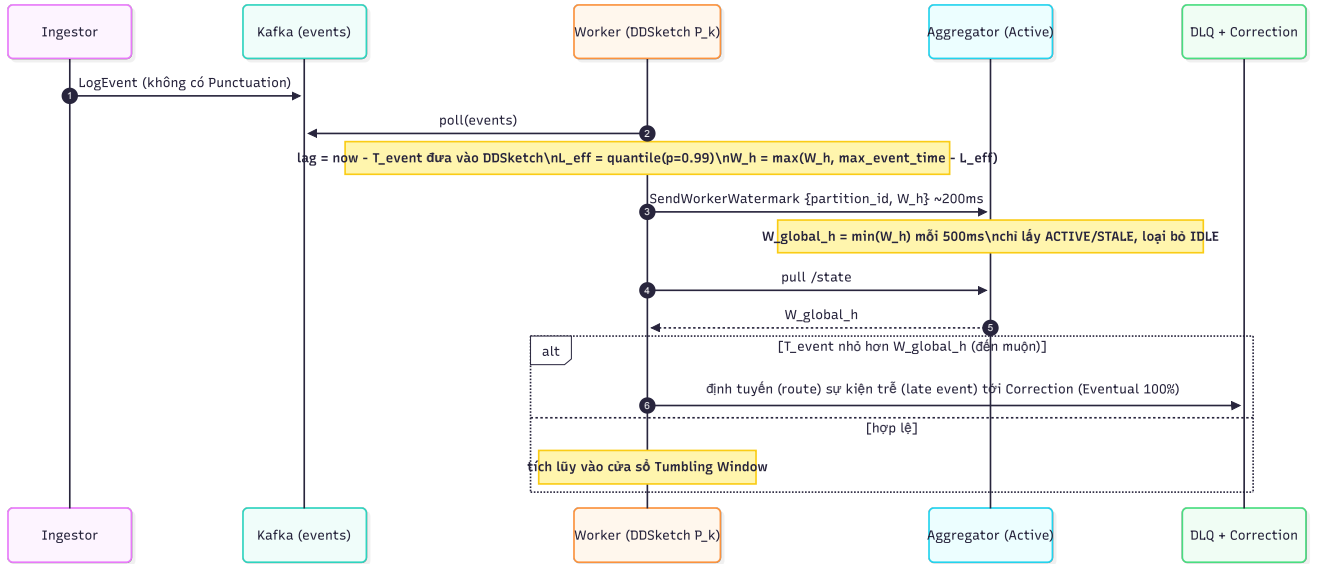
Điều kiện đóng window. Window được đóng khi cuối window nhỏ hơn hoặc bằng W_{global_h} . Vì watermark này là heuristic, kết quả được xem là speculative: nhanh, nhưng có thể cần correction nếu late event xuất hiện. **Điều kiện đưa event vào DLQ.** Event vào DLQ không có nghĩa là bị mất. Nó được giữ lại để correction pipeline tính δ và phát bản sửa cho window đã emit. **Ví dụ vận hành và thuật toán.**

- **Cấu hình hệ thống:** Độ phân vị $p = 0.99$, kích thước cửa sổ 5 giây.
- **Bước 1 (Worker đo $latency$ cục bộ):** Tại partition P0, Worker nhận các bản ghi log và đo độ trễ thực tế. $max_event_time = 200s$. Phác họa $sketch$ thống kê và trả về phân vị 99% của độ trễ là $L_{eff} = 4s$.
- **Bước 2 (Tính watermark cục bộ):** $W_h = max_event_time - L_{eff} = 200s - 4s = 196s$.
- **Bước 3 (Aggregator tổng hợp):** Aggregator nhận báo cáo từ các partition đang hoạt động: P0 = 196s, P1 = 198s, P2 = 194s. Aggregator tính $W_{global_h} = \min(196, 198, 194) = 194s$.
- **Bước 4 (Đóng window speculative):** Worker nhận $W_{global_h} = 194s$. Cửa sổ [185, 190) có $window_end = 190 \leq 194s$ được đóng và phát kết quả tạm tính (speculative result).
- **Bước 5 (Xử lý sự kiện đến muộn qua DLQ):** Một event có $event_time = 192s$ (thuộc cửa sổ [190, 195)) đến muộn khi $now = 206s$ ($latency = 14s$). Lúc này watermark toàn cục đã vượt qua 192s, cửa sổ đã đóng. Bản ghi này được phát hiện là late event, tự động được gửi vào DLQ để pipeline hiệu chỉnh tính toán lại và phát Correction Message sửa đổi kết quả cửa sổ [190, 195).

Cách chạy từng bước.

1. Khi event đến Worker, engine tính $latency$ của event bằng chênh lệch giữa thời điểm xử lý và $event_time$. Mẫu $latency$ này được đưa vào DDSketch của partition.
2. DDSketch giữ phân phối $latency$ theo thang log, nên vẫn mô tả được cả event đến gần như đúng giờ lẫn event trễ rất dài mà không cần histogram khổng lồ.

- Sau giai đoạn warm-up, engine lấy phân vị cấu hình, ví dụ p99, để tạo L_{eff} .
Watermark cục bộ được tính theo $W_h = \max_event_time - L_{eff}$, rồi bị chặn bởi hysteresis/rate limit để không nhảy quá gắt.
- Aggregator nhận W_h từ nhiều partition, bỏ qua partition thật sự idle theo rule của Heuristic, và tính $W_{global_h} = \min(W_h)$ trên phần còn active.
- Worker đóng window theo W_{global_h} . Vì đây là watermark dự đoán, kết quả đầu tiên là kết quả speculative. Nếu sau đó có event thuộc window đã đóng, event được đưa vào DLQ cùng $window_id$, payload gốc và thông tin correction.
- Correction pipeline đọc DLQ, tính phần chênh lệch cần cộng/trừ vào window result, rồi phát correction để kết quả sau cùng vẫn đầy đủ.



Hình 3: Sơ đồ

Liên hệ. Heuristic tái dùng đúng đường dữ liệu của Strict — Kafka + Bounded PQ (6), engine theo partition (5), Tiered Storage (7) — chỉ thay nguồn watermark (DDSketch thay token 3), đổi control plane sang Aggregator (8) và thêm nhánh DLQ. Late event vào DLQ chính là phần Strict (3) không có.

4.2 2.2 Phản biện: Vì sao thiết kế như vậy

Mục Heuristic không cố chứng minh "không bao giờ có late event". Nó chọn latency thấp, nhưng phải nói rõ sai số được đo, được giới hạn và được sửa bằng DLQ/correction. **Phản đề 1: Dùng EWMA hoặc histogram cố định thay DDSketch cho đơn giản.**

- Vì sao nghe hợp lý:** EWMA dễ cài, histogram cố định dễ giải thích, cả hai đều có thể mô tả độ trễ trung bình của luồng log.

- **Lý do bác bỏ:** watermark không cần trung bình, mà cần tail `lateness` như p95/p99 để biết nên chờ bao nhiêu thời gian cho event muộn. EWMA che mất tail; histogram cố định khó chọn bucket vì `lateness` có thể trải qua nhiều bậc độ lớn; Aggregator còn cần merge phân phối từ nhiều partition mà không làm mất ý nghĩa phân vị.
- **Kết luận phản biện:** DDSketch phù hợp hơn vì giữ phân vị với sai số tương đối, merge được giữa partition và dùng bộ nhớ giới hạn.

Phản đề 2: Nếu sợ sai thì cứ chờ đủ, không cần Heuristic.

- **Vì sao nghe hợp lý:** chờ lâu hơn sẽ giảm late event và tránh phải có correction pipeline.
- **Lý do bác bỏ:** chờ đủ chính là mục tiêu của Strict, nhưng Heuristic được thiết kế cho trường hợp upstream không phát token hoặc SLA yêu cầu latency thấp hơn. Nếu cứ chờ như Strict, mode Heuristic mất lý do tồn tại.
- **Kết luận phản biện:** Heuristic được phép chốt sớm, nhưng phải đi kèm DLQ và Correction để late event không bị mất thềm lặng.

Phản đề 3: Partition IDLE vẫn nên nằm trong min() giống Strict.

- **Vì sao nghe hợp lý:** giữ partition idle trong min() có vẻ an toàn hơn vì không bỏ qua nguồn dữ liệu nào.
- **Lý do bác bỏ:** Heuristic không có Empty Punctuation để chứng minh partition idle đã an toàn. Nếu giữ partition rảnh trong min(), watermark heuristic có thể bị treo vô hạn dù các partition còn lại vẫn có dữ liệu mới.
- **Kết luận phản biện:** Heuristic loại partition thật sự idle khỏi min() để giữ latency, còn rủi ro late event được hấp thụ bằng DLQ và correction.

5 3. Thiết kế Node quản lý Partition

5.1 3.1 Trình bày chi tiết thiết kế

Ý chính. Worker Node là nơi chạy xử lý thực tế, nhưng đơn vị quản lý đúng không phải là node mà là partition. Một Worker có thể giữ nhiều partition, ví dụ 3 partition trên một node, nhưng mỗi partition có engine, buffer, state và watermark riêng. Khi gửi heartbeat, Worker gửi bản đồ watermark từng partition thay vì tự gộp thành một watermark của node. **Bản đồ thiết kế. Thành phần và trách nhiệm.**

- **Worker process:** tiến trình chạy nhiều partition trên cùng node.
- **Partition assignment:** danh sách partition mà Worker đang sở hữu.
- Map `partition_id` -> **engine:** mỗi partition trở đến engine xử lý riêng.
- Map `partition_id` -> **buffer:** mỗi partition có buffer event-time riêng.

- **Lock theo partition:** bảo vệ thao tác flush/pause/reassign của từng partition.
- **RocksDB/checkpoint theo partition:** lưu state và `offset` tách biệt để bàn giao độc lập.

Dữ liệu vào.

- **Batch event từ Kafka:** batch chứa record thuộc một hoặc nhiều partition.
- **Partition id:** khóa để dispatch record vào đúng buffer/engine.
- **Lệnh control:** pause, resume, reassign, flush khi failover/failback.

Dữ liệu ra.

- **Local watermark từng partition:** `LWi` trong Strict hoặc `Wh` trong Heuristic.
- **Metric queue/backpressure:** queue size, pause/resume status, lag.
- **Heartbeat per-partition:** trạng thái từng partition gửi lên control plane.

Trạng thái cần giữ.

- **Owner partition:** Worker nào đang sở hữu partition.
- **Buffer event-time:** event đang chờ sắp/xử lý của từng partition.
- **Engine state:** open window, aggregate, watermark cục bộ.
- **Backpressure flag:** trạng thái pause/resume consume.
- **Last event time:** dùng để nhận diện partition idle.

Giao tiếp.

- **Worker -> Kafka:** poll event theo partition được assign.
- **Partition engine -> Worker:** trả watermark/metric cục bộ.
- **Worker -> Coordinator/Aggregator:** gửi heartbeat dạng map theo partition.
- **Control plane -> Worker:** gửi lệnh pause, flush, reassign hoặc resume.

Đầu ra.

- **Heartbeat chi tiết:** trạng thái từng partition, không phải trạng thái gộp của node.
- **Window result:** kết quả sinh từ engine của từng partition.

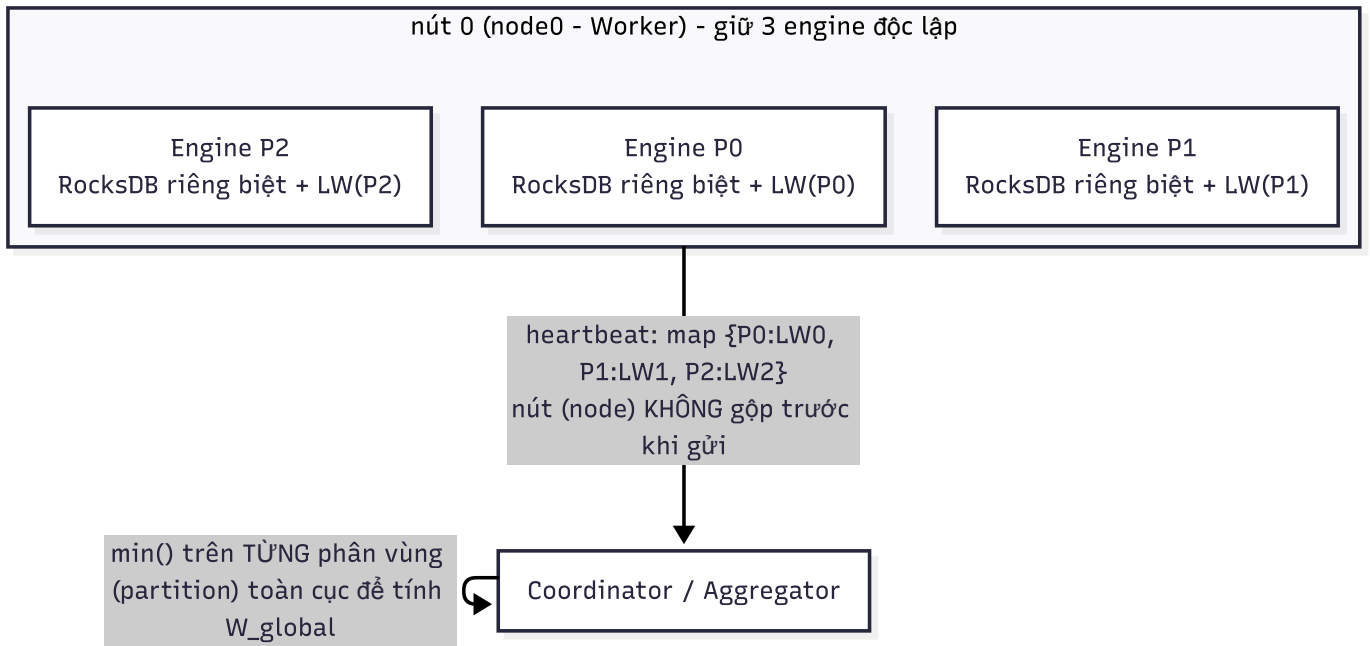
Cách chạy từng bước.

1. Coordinator gán một tập partition cho Worker. Worker tạo engine, buffer, lock và state store riêng cho từng partition trong tập đó.

2. Khi poll được batch từ Kafka, Worker nhìn `partition_id` của từng record để dispatch vào đúng buffer. Event của partition P3 không được đi vào engine của P4, dù hai partition đang nằm trên cùng một process Worker.
3. Mỗi partition xử lý độc lập: buffer sắp event, engine cập nhật window, local watermark tăng theo mode Strict hoặc Heuristic, checkpoint ghi theo partition.
4. Heartbeat của Worker là một map chi tiết, ví dụ P0: `watermark/offset/status`, P1: `watermark/offset/status`, P2: `watermark/offset/status`. Control plane nhìn được partition nào chậm, partition nào idle, partition nào cần chuyển chủ.
5. Khi một partition được reassign, Worker chỉ pause/flush/close state của partition đó. Các partition khác trên cùng node vẫn có thể tiếp tục chạy, miễn không dùng chung lock hoặc RocksDB instance gây kẹt.

Ví dụ vận hành.

- **Bố trí ban đầu:** Hệ thống gồm 2 Worker: W1 (gánh P0, P1, P2) và W2 (gánh P3, P4, P5).
- **Trường hợp 1 (Nhiều partition độc lập trên cùng node):** Partition P1 trên W1 bị nghẽn (do Kafka partition P1 có burst lượng log lớn), khiến local watermark của P1 đứng yên tại 100. Các partition P0 và P2 trên W1 vẫn chạy mượt và tăng local watermark lên 120. Nhờ heartbeat báo cáo chi tiết dạng map P0: 120, P1: 100, P2: 120, Coordinator biết chính xác chỉ P1 bị chậm, P0 và P2 vẫn bình thường.
- **Trường hợp 2 (Failover cô lập):** Nếu P1 trên W1 bị lỗi nghiêm trọng về RocksDB local, Coordinator chỉ cần thực hiện failover chuyển quyền sở hữu riêng partition P1 sang W2. Tiến trình W1 không bị khởi động lại hay tạm dừng toàn bộ; các partition P0 và P2 trên W1 tiếp tục tiêu thụ dữ liệu và cập nhật bình thường mà không bị ảnh hưởng.



Hình 4: Sơ đồ

Liên hệ. Watermark mà mỗi engine sinh ra ($3 \frac{LW_i}{4} W_h$) được node gom thành *map* rồi gửi control plane (8) qua kênh control (9). Vì đơn vị là partition nên khi node chết, chỉ partition đó được bàn giao qua Failback (10) dựa trên checkpoint per-partition (7).

5.2 3.2 Phản biện: Vì sao thiết kế như vậy

Mục Worker/Partition phải bảo vệ nguyên tắc: partition là đơn vị sở hữu, xử lý, checkpoint và failover. Node chỉ là nơi đang chạy partition ở thời điểm hiện tại. **Phản đề 1: Một Worker dùng chung một engine và một RocksDB cho mọi partition.**

- **Vì sao nghe hợp lý:** ít object hơn, ít thư mục RocksDB hơn, code quản lý state có vẻ gọn hơn.
- **Lý do bác bỏ:** khi failover chỉ một partition, hệ thống cần flush, đóng khóa, checkpoint và bàn giao đúng partition đó. Nếu nhiều partition dùng chung engine/state store, việc chuyển riêng một partition sẽ kéo theo state của partition khác, dễ tranh chấp lock và khó xác định *offset* an toàn theo partition.
- **Kết luận phản biện:** mỗi partition cần engine, buffer, lock và state riêng để có thể chuyển giao độc lập mà không dừng cả node.

Phản đề 2: Worker gộp watermark theo node rồi mới gửi lên control plane.

- **Vì sao nghe hợp lý:** heartbeat nhỏ hơn, Coordinator/Aggregator xử lý ít key hơn, nhìn bề ngoài hệ thống đơn giản hơn.

- **Lý do bác bỏ:** watermark theo node che mắt khác biệt giữa các partition. Một partition chậm sẽ kéo lùi oan các partition khỏe; ngược lại, nếu gộp không cẩn thận, partition chậm có thể bị che khuất và gây late/loss. Control plane cũng không biết partition nào cần failover hoặc đang idle.
- **Kết luận phản biện:** heartbeat phải giữ map theo partition. Gộp theo node chỉ nên dùng cho metric tổng hợp, không dùng cho quyết định watermark và ownership.

6 4. Thiết kế Kafka + Bounded Priority Queue tại Node

6.1 4.1 Trình bày chi tiết thiết kế

Ý chính. Kafka là data plane bền vững của hệ thống. Event được ghi vào Kafka theo partition và `offset` để Worker có thể đọc lại sau sự cố. Tuy nhiên Kafka chỉ giữ thứ tự append theo `offset` trong partition, không sắp xếp lại theo event-time. Vì vậy mỗi Worker đặt thêm một BoundedPriorityQueue, triển khai như min-heap theo event-time, ở phía client trước khi event đi vào window engine. **Bản đồ thiết kế. Thành phần và trách nhiệm.**

- **Ingestor/KafkaProducer:** ghi event vào Kafka theo key/partition.
- **Kafka topic events:** lưu event bền vững theo partition và `offset`.
- **Kafka consumer ở Worker:** poll batch event theo partition được assign.
- **BoundedPriorityQueue:** min-heap giới hạn kích thước/thời gian, sắp event theo event-time.
- **Window engine:** nhận event đã qua queue và cập nhật window state.

Quan hệ giữa Kafka, Worker node và partition. Trong thiết kế này cần phân biệt rõ Kafka partition và partition xử lý do node quản lý. Hai khái niệm này dùng cùng một `partition_id`, nhưng trách nhiệm khác nhau:

Thành phần	Nằm đâu	ở	Gắn với node không?	Gắn với partition không?	Vai trò trong thiết kế
Kafka topic events	Kafka broker		Không gắn cố định với Worker node	Có, theo Kafka partition	Lưu log event bền vững theo <i>offset</i>
Worker node	Cụm xử lý		Có, là process/máy đang chạy	Có, nhận một tập partition được giao	Đọc event, sắp event-time, cập nhật state
Consumer assignment	Coordinator/group	+	Consumer xác định node nào đang xử lý	Có, assign theo partition	Ràng buộc <i>partition_id</i> - <i>worker_id</i> tại thời điểm hiện tại
Kafka <i>offset</i> /checkpoint	Kafka Broker-2 checkpoint	+	Không nên gắn vĩnh viễn với node	Có, <i>offset</i> luôn theo topic-partition	Biết replay từ đâu khi recovery/failover
BoundedPriorityQueue và window engine	Queue nhớ/state local của Worker		Có, nằm trên owner node hiện tại	Có, tách riêng theo partition	Sắp out-of-order và xử lý window cho partition đó

Ví dụ topic `events` có 12 partition `P0..P11` và có 4 Worker node `W1..W4`. Coordinator có thể gán:

- W1 quản lý P0, P1, P2.
- W2 quản lý P3, P4, P5.
- W3 quản lý P6, P7, P8.
- W4 quản lý P9, P10, P11.

Kafka vẫn là nơi lưu toàn bộ log của `P0..P11`. Worker node không sở hữu dữ liệu Kafka; Worker chỉ là **consumer/processor owner** của những partition được giao. Khi nói W2 quản lý P4, nghĩa là tại thời điểm đó W2 là node duy nhất được phép consume, buffer, tính watermark, cập nhật window state và checkpoint *offset* cho P4. Luồng quan hệ cụ thể như sau:

1. Ingestor ghi event vào Kafka topic `events` với key/partition phù hợp. Kafka đặt record vào đúng Kafka partition, ví dụ P4, và cập *offset* tăng dần trong P4.
2. Coordinator hoặc cơ chế consumer assignment biết P4 đang thuộc W2, nên chỉ W2 được poll record của P4.

3. Bên trong W2, record của P4 không đi vào hàng đợi chung của cả node. Nó được đưa vào BoundedPriorityQueue riêng của P4, sau đó đi vào window engine/state riêng của P4.
4. Sau khi xử lý an toàn, W2 ghi checkpoint cho P4, gồm Kafka `offset` đã xử lý, watermark, window state và metadata cần phục hồi.
5. Nếu W2 lỗi và P4 chuyển sang W1, W1 không đọc state theo tên W2. W1 đọc checkpoint theo `partition_id` = P4, khôi phục PQ/state cần thiết, rồi seek Kafka partition P4 từ `offset` an toàn kế tiếp.

Dữ liệu vào.

- **Event key/partition:** xác định event thuộc partition nào.
- **Kafka `offset`:** vị trí bền vững để replay khi recovery.
- **Event-time:** thời điểm sự kiện dùng để sắp heap và tính window.
- **Poll batch:** nhóm record Worker lấy từ Kafka.

Dữ liệu ra.

- **Phần tử heap:** tuple (event_time, counter, event).
- **Event đã pop:** event được đưa sang engine theo thứ tự event-time tương đối.
- **Offset đã checkpoint:** `offset` an toàn sau khi event được xử lý.

Trạng thái cần giữ.

- **Kafka `offset` hiện tại:** `offset` đã poll và `offset` đã checkpoint.
- **Heap buffer:** event đang chờ sắp/xả.
- **Queue size:** số event trong heap để quyết định backpressure.
- **Backpressure pause/resume:** trạng thái tạm dừng hoặc tiếp tục consume Kafka.

Giao tiếp.

- **Ingestor -> Kafka:** produce event.
- **Worker -> Kafka:** poll event và pause/resume consumer khi cần.
- **Queue -> Engine:** pop event nhỏ nhất theo event-time sang window engine.
- **Engine -> Checkpoint:** ghi `offset` /state sau khi xử lý.

Đầu ra.

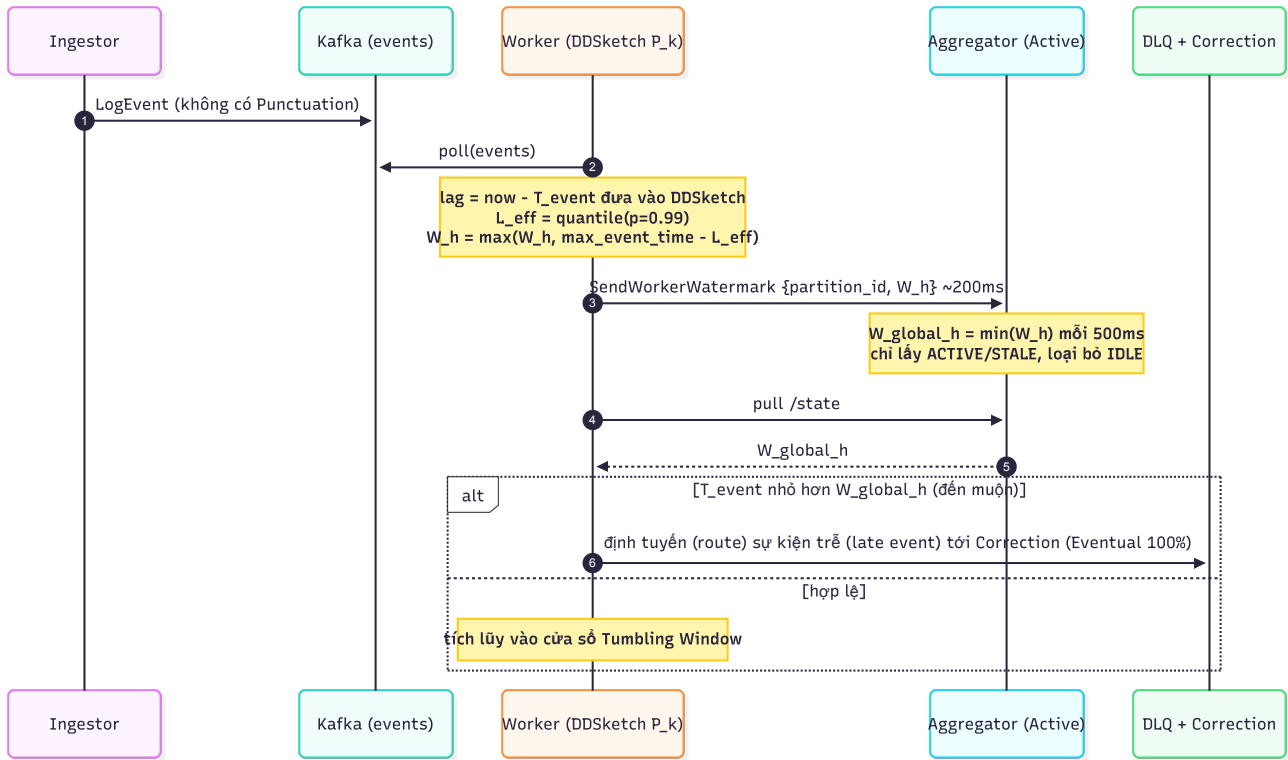
- **Event vào engine:** đã được sắp xếp theo event-time.
- **Tín hiệu backpressure:** giúp Worker không tràn RAM khi event đến nhanh hơn tốc độ xử lý.

Cách chạy từng bước.

1. Ingestor ghi event vào Kafka topic **events**, key theo partition để các event cùng partition giữ thứ tự `offset` và có thể replay.
2. Worker consumer poll batch theo partition. Record mới poll chưa được đưa thẳng vào window engine, mà được đưa vào BoundedPriorityQueue.
3. Queue là min-heap theo **event_time**, có thêm counter để phá hòa khi hai event cùng timestamp. Nhờ vậy, nếu Kafka đưa ra `offset` theo thứ tự **event_time**=100, 95, 98, Worker có thể xử lý theo thứ tự gần đúng 95, 98, 100.
4. Queue bị giới hạn bởi **maxsize** và `max_wait_ms`. Khi queue đầy, phần tử nhỏ nhất được pop sang engine để giải phóng RAM. Khi event nằm trong queue quá lâu, nó cũng được xả để tránh chờ vô hạn.
5. Nếu queue thường xuyên đầy, Worker pause Kafka consumer tạm thời. Kafka giữ log trên broker, còn Worker xử lý bớt heap và resume sau. Offset chỉ nên được xem là an toàn khi event tương ứng đã đi qua engine và state/`offset` đã được checkpoint.

Ví dụ vận hành.

- **Luồng dữ liệu out-of-order:** Kafka partition P4 nhận được 3 event theo thứ tự `offset` như sau:
 - Offset 10 mang **event_time** = 12:00:10
 - Offset 11 mang **event_time** = 12:00:02
 - Offset 12 mang **event_time** = 12:00:05
- **Bước 1 (Poll dữ liệu):** Worker poll batch này từ Kafka broker.
- **Bước 2 (Đưa vào Bounded PQ):** Thay vì xử lý trực tiếp, Worker đẩy cả 3 event vào BoundedPriorityQueue riêng của P4. Queue sắp xếp chúng lại theo thứ tự thời gian sự kiện tăng dần (min-heap).
- **Bước 3 (Xả dữ liệu sắp xếp):** Khi queue đạt điều kiện xả (hoặc đầy hoặc hết thời gian chờ), các event được pop ra theo thứ tự: 12:00:02 (Offset 11) trước, rồi đến 12:00:05 (Offset 12), cuối cùng mới là 12:00:10 (Offset 10).
- **Bước 4 (Kết quả):** Window engine nhận được luồng event đã được sắp xếp lại gần như tuần tự hoàn hảo theo `Tevent`, giúp watermark của P4 không bị nhảy giật cục bộ và tránh đóng nhầm cửa sổ.



Hình 5: Sơ đồ

Liên hệ. Event sau khi qua Bounded PQ mới vào engine từng partition (5) để tính watermark (3/4). Offset Kafka được checkpoint xuống Tier-2 (7) và là mốc `seek(offset + 1)` khi Failback (10). Kafka cũng chờ luồng kết quả/DLQ (4) và ingestor-heartbeat (9).

6.2 4.2 Phản biện: Vì sao thiết kế như vậy

Mục Kafka + Bounded Priority Queue phải trả lời hai câu hỏi riêng: dữ liệu được giữ bên ở đâu, và out-of-order theo event-time được xử lý ở đâu. **Phản đề 1: Ingestor đẩy HTTP trực tiếp vào Worker, bỏ Kafka.**

- **Vì sao nghe hợp lý:** ít thành phần hạ tầng hơn, đường đi ngắn hơn, dễ nhìn thấy request/response.
- **Lý do bác bỏ:** nếu Worker sập trong lúc nhận hoặc xử lý request, dữ liệu đang bay có thể mất. Không có `offset` bên thì worker mới không biết replay từ đâu. Hệ thống cũng mất khả năng pause/resume bằng backpressure vì Ingestor phải tự giữ dữ liệu.
- **Kết luận phản biện:** Kafka cần thiết vì nó là log bền vững, có partition, `offset` và khả năng replay sau sự cố.

Phản đề 2: Kafka đã có thứ tự partition, không cần sort ở Worker.

- **Vì sao nghe hợp lý:** Kafka bảo đảm thứ tự record trong một partition theo `offset`, nên có vẻ đã đủ để xử lý tuần tự.

- **Lý do bác bỏ:** thứ tự `offset` không đồng nghĩa với thứ tự event-time. Bài toán window tính theo `Tevent`; nếu event có `Tevent` cũ đến sau `offset` mới, engine vẫn cần một lớp sắp xếp tương đối theo event-time trước khi cập nhật window.
- **Kết luận phản biện:** Kafka giữ durability và replay; BoundedPriorityQueue xử lý out-of-order theo event-time ở phía Worker. Hai thành phần giải hai vấn đề khác nhau.

Phản đề 3: Dùng min-heap không giới hạn để giữ đúng event-time hơn.

- **Vì sao nghe hợp lý:** giữ càng nhiều event thì càng có cơ hội sắp đúng theo event-time.
- **Lý do bác bỏ:** luồng streaming không có điểm kết thúc tự nhiên; heap không giới hạn sẽ phình RAM và có thể làm Worker chết. Nếu chờ quá lâu để đủ thứ tự tuyệt đối, latency cũng tăng vô hạn.
- **Kết luận phản biện:** heap phải bounded bằng maxsize và `max_wait_ms` để cân bằng giữa sắp event-time, giới hạn bộ nhớ và tiến độ xử lý.

7 5. Thiết kế Tiered Storage

7.1 5.1 Trình bày chi tiết thiết kế

Ý chính. Tiered Storage chia state theo vòng đời và yêu cầu truy cập. Tier-1 là RocksDB cục bộ để phục vụ hot path cập nhật từng event. Tier-2 là Shared Volume hoặc checkpoint directory để phục hồi nhanh khi worker crash hoặc partition chuyển chủ. Tier-3 là MinIO/Object Storage để lưu closed window, archive và disaster recovery. Ba tầng vận hành theo ba nhịp độc lập: xử lý nóng, checkpoint định kỳ và archive dài hạn. **Bản đồ thiết kế. Thành phần và trách nhiệm.**

- **Tier-1 RocksDB local:** giữ hot state đang được cập nhật theo từng event.
- **Tier-2 checkpoint directory/Shared Volume:** giữ checkpoint định kỳ để worker khác phục hồi nhanh.
- **Tier-3 MinIO bucket:** giữ closed window, archive và backup phục vụ DR.
- **EvictionManager:** điều khiển vòng đời closed window từ local sang object storage rồi purge.
- **TieredStorageManager:** gom logic ghi/đọc giữa ba tier và cung cấp API phục hồi state.

Quan hệ giữa node, partition và tier.

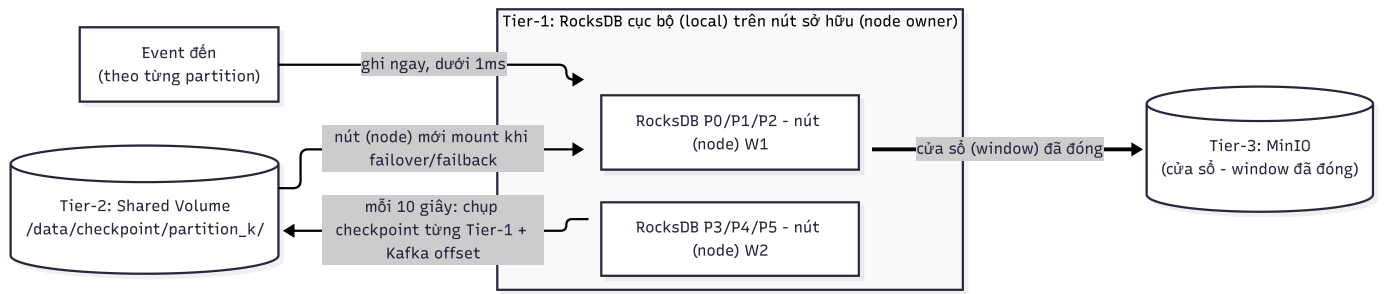
- **Worker node:** là máy/process đang chạy xử lý tại một thời điểm. Một node có thể giữ nhiều partition, ví dụ W1 giữ P0, P1, P2.

- **Partition:** là đơn vị sở hữu state. Mọi state quan trọng phải gắn với `partition_id`, không gắn cố định với `worker_id`.
- **Tier-1:** nằm local trên node đang là owner của partition. Nếu P1 đang chạy trên W1, hot state của P1 nằm trong RocksDB local của W1.
- **Tier-2:** nằm ngoài node hiện tại hoặc ở shared checkpoint path. Checkpoint của P1 được định danh theo `partition_id`, nên node khác vẫn đọc được khi P1 chuyển owner.
- **Tier-3:** nằm trên object storage dùng chung. Closed window/archive của P1 cũng dùng key theo partition/window, không phụ thuộc node nào đã tạo ra nó.

Bảng quan hệ ownership và lưu trữ.

Đối tượng	Gắn với node?	Gắn với partition?	Vai trò trong Tiered Storage
Worker node	Có, nhưng chỉ tạm thời	Không phải định danh state chính	Chạy engine và giữ Tier-1 local cho các partition đang sở hữu
Partition	Có thể đổi node	Có, là định danh chính	Quyết định namespace state, checkpoint, <code>offset</code> và object key
Tier-1 RocksDB local	Có, nằm trên node owner hiện tại	Có, tách theo partition	Hot state nhanh, mất node thì phải phục hồi từ Tier-2
Tier-2 checkpoint	Không phụ thuộc một node cụ thể	Có, key theo partition/checkpoint version	Nơi node mới đọc để nhận partition sau failover/failback
Tier-3 MinIO/Object Storage	Không phụ thuộc node	Có, key theo partition/window id	Lưu closed window, archive và backup DR dài hạn

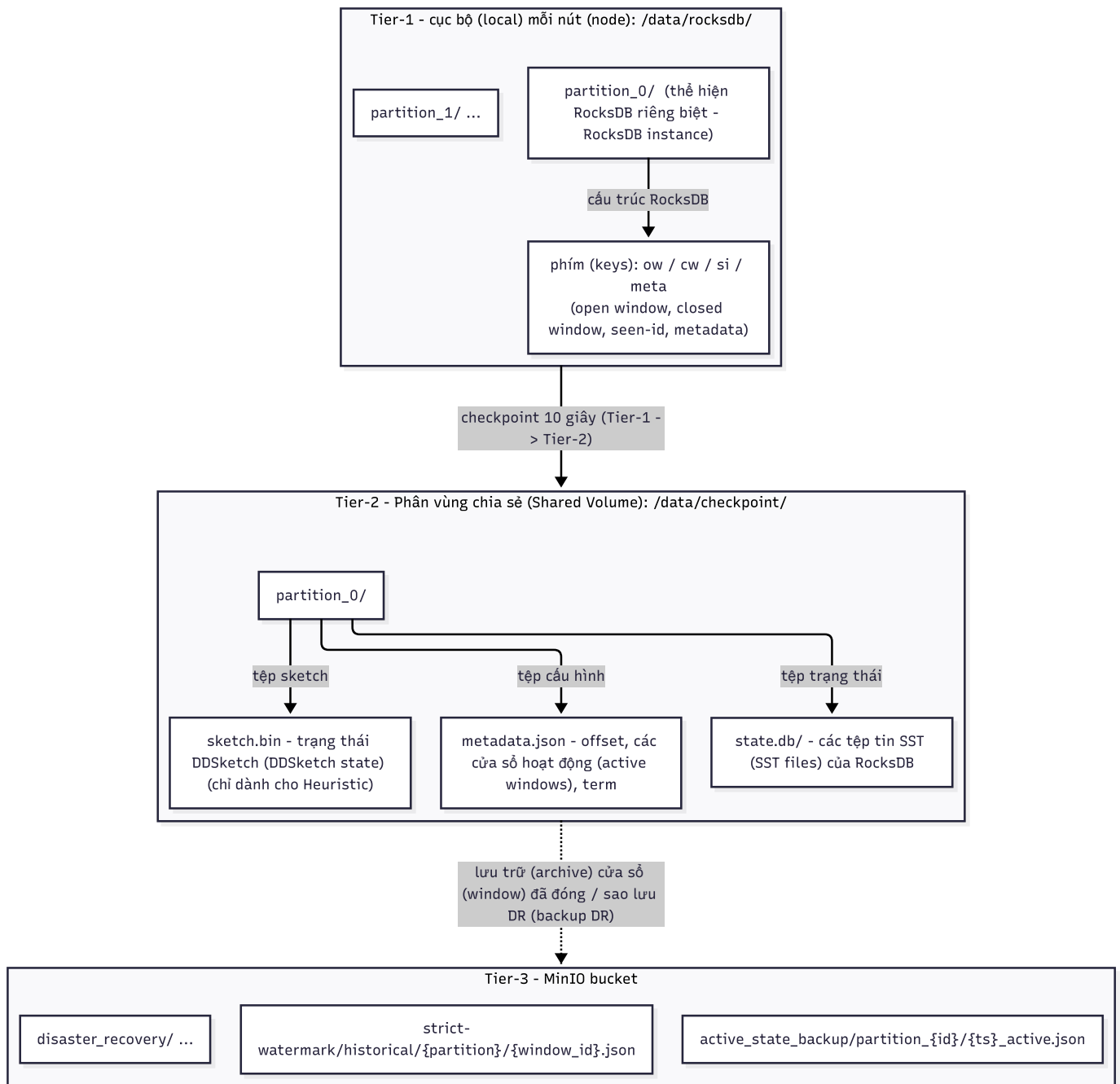
Luồng lưu trữ: Tier-1 nhận event, Tier-2 checkpoint Tier-1 vào Shared Volume. Cách hình dung đơn giản nhất: event của mỗi partition được ghi *ngay* vào Tier-1 — RocksDB cục bộ nằm trên node đang sở hữu partition đó. Định kỳ 10 giây, Tier-2 **chụp lại nội dung Tier-1** của từng partition (kèm Kafka `offset` an toàn) và lưu sang Shared Volume dùng chung. Khi node chết, node mới chỉ cần mount checkpoint của partition trên Shared Volume rồi chạy tiếp; checkpoint đặt tên theo `partition_id` nên không phụ thuộc node cũ. Window đã đóng được đẩy tiếp lên Tier-3 để lưu dài hạn.



Hình 6: Sơ đồ luồng (Tier-1 → Tier-2 → Tier-3)

Cấu trúc thư mục. Để dễ hình dung state nằm ở đâu, đây là bố cục thư mục của 3 tầng:

- **Tier-1** — cục bộ mỗi node, `/data/rocksdb/`: mỗi partition một thư mục RocksDB **riêng** (`partition_k/`); bên trong tách namespace bằng tiền tố key: **ow**: (window đang mở), **cw**: (window đã đóng), **si**: (event-id đã thấy, chống trùng), **meta**: (`offset`/watermark).
- **Tier-2** — Shared Volume, `/data/checkpoint/`: mỗi partition một thư mục checkpoint chứa `metadata.json` (`offset`, danh sách window, `term`), `state.db`/ (các file SST của RocksDB) và `sketch`.bin (state DDSketch — chỉ Heuristic).
- **Tier-3** — MinIO: object key theo partition/window cho window đã đóng (`historical/`), backup DR (`active_state_backup/`) và `disaster_recovery/`.



Hình 7: Sơ đồ cấu trúc thư mục

Khi partition chuyển node.

1. Coordinator quyết định P1 chuyển từ W1 sang W2.
2. W1 dừng xử lý P1, flush Tier-1 của P1 ra Tier-2 kèm *offset* an toàn.
3. W2 tạo hoặc mở RocksDB local mới cho P1.
4. W2 đọc checkpoint của P1 từ Tier-2, khôi phục open window, dedup state và watermark.
5. W2 seek Kafka theo *offset* trong checkpoint rồi tiếp tục xử lý P1.

6. Các closed window trước đó của P1 vẫn nằm ở Tier-3, vì object key gắn với partition/window chứ không gắn với W1.

Dữ liệu ở Tier-1.

- **Open window:** aggregate của window chưa đóng.
- **Dedup ids:** id đã xử lý để tránh replay trùng.
- **Kafka `offset` gần nhất:** `offset` đã xử lý/chờ checkpoint.
- **Local engine state:** watermark, buffer metadata và trạng thái engine.

Dữ liệu ở Tier-2.

- **Checkpoint snapshot:** bản chụp state theo chu kỳ.
- **SST manifest:** danh sách file/state cần mở lại.
- **Offset an toàn:** `offset` dùng để `seek(offset + 1)` khi worker mới nhận partition.
- **Failback state:** trạng thái bàn giao partition nếu đang failback.

Dữ liệu ở Tier-3.

- **Closed window:** window result đã đóng và cần lưu dài hạn.
- **Archive result:** dữ liệu phục vụ audit/đối soát.
- **Backup DR:** backup window/state theo chu kỳ dài hơn để giảm RPO.
- **Object key:** khóa object theo partition/`window_id` để upload idempotent.

Luồng ghi.

- **Ghi nóng:** từng event cập nhật Tier-1 ngay.
- **Checkpoint:** timer 10s chụp state từ Tier-1 sang Tier-2 kèm `offset`.
- **Upload closed window:** khi `window_end ≤ Wglobal`, window đóng được upload lên Tier-3.
- **Backup DR:** timer 5 phút backup window đang mở lên Tier-3.

Luồng đọc/phục hồi.

- **Failover nhanh:** worker mới đọc Tier-2, mở lại state partition và seek Kafka từ `offset + 1`.

- **Audit/restore xa:** hệ thống đọc object Tier-3 khi cần dữ liệu closed window hoặc DR.
- **Retry upload:** nếu upload Tier-3 lỗi, dữ liệu vẫn giữ ở Tier-1/Tier-2 để thử lại.

Trạng thái vòng đời.

- **CLOSED:** window đã đủ điều kiện đóng.
- **UPLOADING:** window đang được upload lên Tier-3.
- **UPLOADED:** upload thành công, object đã tồn tại.
- **PURGED:** bản local đã được xóa sau khi upload an toàn.

Giao tiếp.

- **Worker -> RocksDB:** ghi/đọc hot state cục bộ.
- **Worker -> Tier-2:** ghi checkpoint định kỳ và đọc checkpoint khi recovery.
- **Worker -> MinIO:** upload closed window/backup qua client S3.
- **Worker -> Kafka:** sau khi đọc checkpoint, seek về *offset* an toàn để replay phần còn thiếu.

Đầu ra.

- **State nóng:** còn ở Tier-1.
- **Checkpoint phục hồi:** nằm ở Tier-2.
- **Closed window/archive:** nằm ở Tier-3.
- **Recovery point:** cặp state + *offset* để worker mới tiếp tục xử lý.

Bản đồ 3 tier.

Tier	Khi nào ghi	Dữ liệu chính	Khi nào đọc lại	Vai trò
Tier-1 RocksDB local	Ghi liên tục khi từng event được xử lý	Open window, aggregate tạm thời, dedup id, <i>offset</i> gần nhất, local engine state	Đọc ngay trên hot path khi event tiếp theo cùng window đến	Tốc độ thấp độ trễ, phục vụ cập nhật từng event
Tier-2 checkpoint/shared volume	Ghi theo timer 10s hoặc trước khi bàn giao partition	Snapshot state, SST manifest, <i>offset</i> an toàn, owner/epoch, failback state	Worker mới đọc khi failover/failback để mở lại partition	Phục hồi nhanh khi worker chết hoặc partition đổi owner
Tier-3 MinIO/Object Storage	Ghi khi window đã đóng, hoặc backup window đang mở theo timer dài hơn	Closed window, archive result, backup DR, object key theo partition/window	Đọc khi cần audit, restore xa hơn, hoặc Tier-2 không đủ	Lưu dài hạn và disaster recovery

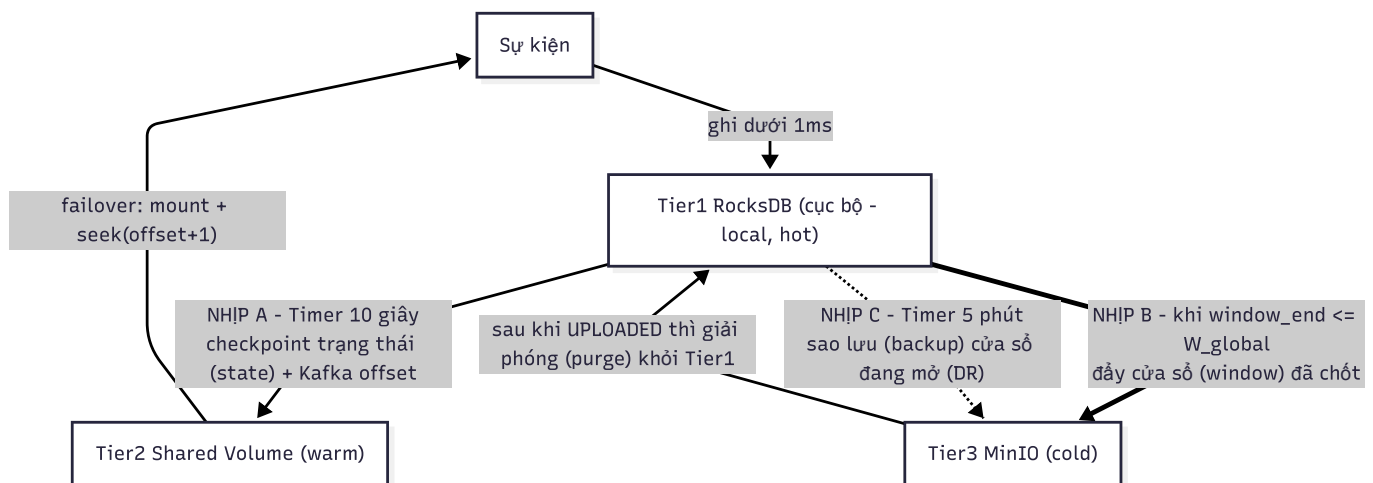
Cách 3 tier vận hành từng bước.

1. Event mới luôn đi vào Tier-1 trước. Engine cập nhật open window trong RocksDB vì thao tác này xảy ra với từng event và cần độ trễ thấp.
2. Sau mỗi chu kỳ checkpoint, Worker chụp state đủ an toàn từ Tier-1 sang Tier-2. Checkpoint phải đi kèm *offset* Kafka; nếu chỉ có state mà không có *offset* thì recovery sẽ không biết đọc tiếp từ đâu.
3. Khi watermark cho phép đóng window, window chuyển sang trạng thái CLOSED. Worker upload kết quả và metadata lên Tier-3, chuyển trạng thái sang UPLOADING, rồi UPLOADED.
4. Chỉ sau khi upload Tier-3 thành công, Worker mới được purge bản local để giảm dung lượng Tier-1. Nếu upload lỗi, state vẫn giữ ở Tier-1/Tier-2 và retry, tránh mất closed window.
5. Khi worker chết, worker mới đọc Tier-2 để phục hồi nhanh, mở lại RocksDB/state, rồi seek(*offset* + 1) trên Kafka để replay phần chưa chắc chắn. Nếu mất cả môi

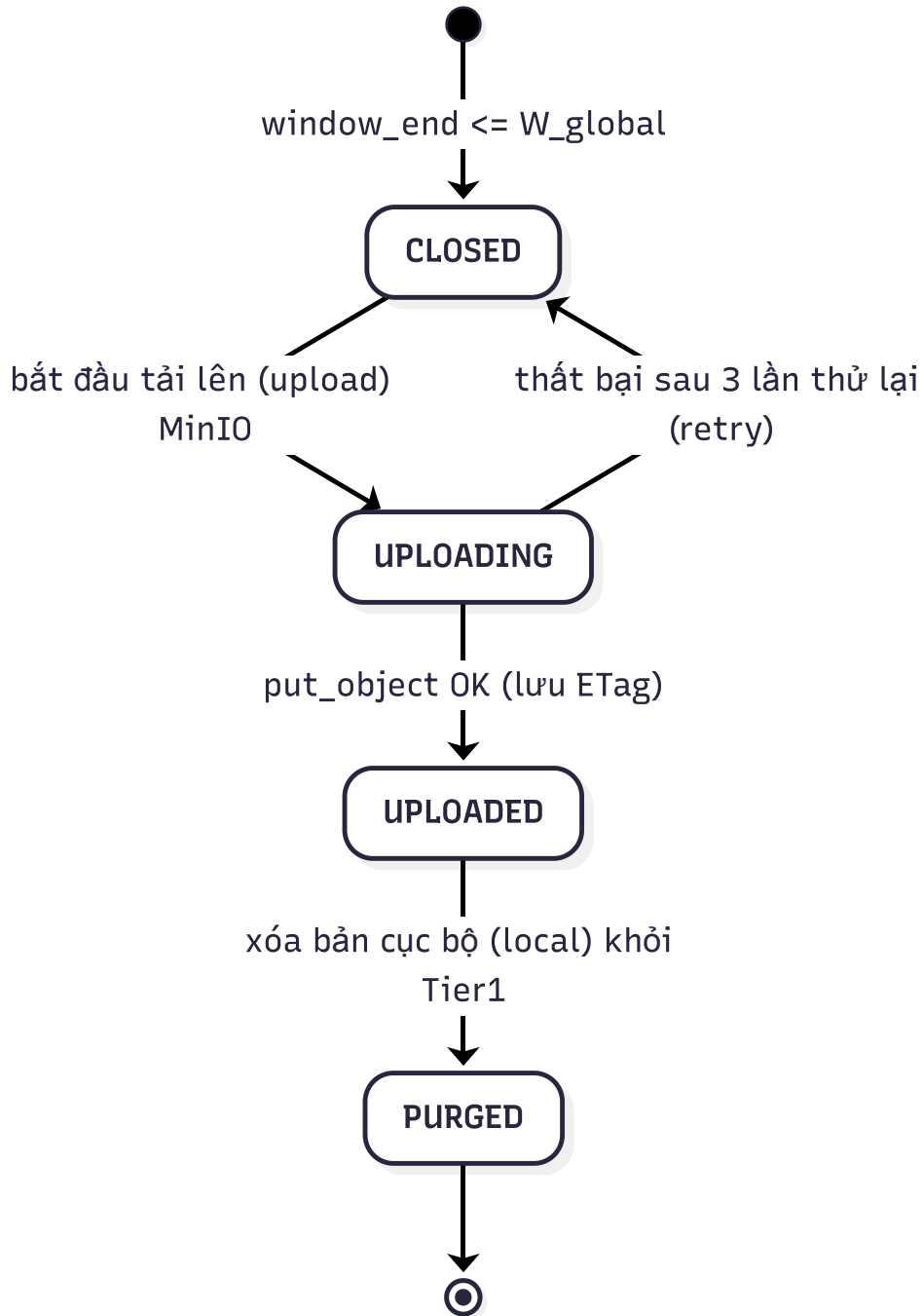
trường cục bộ hoặc cần audit dài hạn, hệ thống dùng Tier-3.

Ví dụ vận hành.

- **Vòng đời của cửa sổ [10:00:00, 10:00:05) trên partition P1:**
- **Giai đoạn 1 (Ghi nóng vào Tier-1):** Khi các event có event-time trong khoảng [10:00:00, 10:00:05) được xử lý, aggregate tạm thời (ví dụ: $\text{sum}(\text{sales}) = 1500$) được ghi trực tiếp vào RocksDB cục bộ của Worker đang chạy P1 (Tầng Tier-1). Thao tác đọc/ghi cực nhanh ($< 1\text{ms}$).
- **Giai đoạn 2 (Checkpoint sang Tier-2):** Đúng chu kỳ 10 giây, Worker thực hiện flush state của P1 từ Tier-1 ghi thành file SST và lưu vào Shared Volume /data/checkpoint/partition_1/ (Tầng Tier-2) kèm theo mốc *offset* Kafka an toàn đã xử lý là 500.
- **Giai đoạn 3 (Đóng và đẩy lên Tier-3):** Khi watermark toàn cục vượt qua 10:00:05, cửa sổ này được đóng. Kết quả chính thức được Worker đẩy lên MinIO bucket (Tầng Tier-3) với object key `historical/P1/10:00:00-10:00:05.json`. Sau khi MinIO xác nhận lưu trữ thành công, bản ghi cục bộ của cửa sổ này trong RocksDB được giải phóng (evicted/purged) để tiết kiệm ổ đĩa cục bộ.
- **Trường hợp sập nguồn:** Nếu Worker bị crash đột ngột ở Offset 505, Worker gán hộ mới được Coordinator chỉ định. Worker mới chỉ cần đọc checkpoint từ /data/checkpoint/partition_1/ (Tier-2) để khôi phục nhanh state tại Offset 500, sau đó seek consumer Kafka về *offset* 501 và replay 5 event tiếp theo để xử lý tiếp mà không phải quét lại từ đầu luồng Kafka.



Hình 8: Sơ đồ luồng 3 tầng



Hình 9: Sơ đồ vòng đời window đã chốt

Liên hệ. State ở 3 tầng đến từ engine mỗi partition (5); checkpoint Tier-2 chính là thứ Failback (10) đọc lại để node mới tiếp quản; `offset` đi kèm checkpoint là mốc replay Kafka (6). Đường ghi/đọc storage là một trong các kênh ở 9.

7.2 5.2 Phản biện: Vì sao thiết kế như vậy

Mục Tiered Storage phải phản biện rõ vì sao không thể dùng một tầng lưu trữ duy nhất. Ba yêu cầu của state là khác nhau: hot path phải nhanh, failover phải phục hồi được, archive/DR phải bền dài hạn. **Phản đề 1: Chỉ dùng Tier-1 RocksDB local.**

- **Vì sao nghe hợp lý:** RocksDB rất nhanh, có WAL, phù hợp cập nhật từng event và đơn giản hơn so với ba tầng.
- **Lý do bác bỏ:** RocksDB local gắn với node đang chạy. Khi node chết, node khác không thể chắc chắn mở cùng state ngay lập tức; thư mục RocksDB còn có lock độc quyền và có thể nằm trên disk cục bộ đã mất. Nếu chỉ có Tier-1, failover không có checkpoint bền để worker mới đọc lại.
- **Kết luận phản biện:** Tier-1 chỉ phù hợp hot path, không đủ để bảo vệ recovery khi worker hoặc máy chứa state chết.

Phản đề 2: Dùng Tier-1 + Tier-2 là đủ, không cần Tier-3.

- **Vì sao nghe hợp lý:** Tier-2 đã có checkpoint để failover, nên nhìn qua có vẻ đã giải quyết được sự cố worker chết.
- **Lý do bác bỏ:** Tier-2 thường là shared volume/checkpoint gần hệ thống runtime; nó phục vụ phục hồi nhanh, không phải lưu trữ dài hạn hoặc DR. Nếu shared volume lỗi, checkpoint của nhiều partition có thể mất cùng lúc. Nó cũng không phải nơi tốt để giữ closed window/archive lâu dài.
- **Kết luận phản biện:** Tier-2 giải bài toán failover nhanh, nhưng chưa giải bài toán lưu lâu dài, audit và disaster recovery.

Phản đề 3: Chỉ dùng Tier-3 MinIO/Object Storage cho mọi state.

- **Vì sao nghe hợp lý:** MinIO/S3 bền, dùng chung giữa node, không phụ thuộc disk cục bộ.
- **Lý do bác bỏ:** object storage có latency cao hơn nhiều so với RocksDB local và không phù hợp cập nhật nhỏ theo từng event. Nếu mọi open window đều đọc/ghi qua MinIO, hot path sẽ chậm, tốn băng thông và khó đạt latency xử lý streaming.
- **Kết luận phản biện:** Tier-3 phù hợp archive/DR, không phù hợp làm state store nóng cho từng event.

Kết luận chung: ba tier không phải thêm cho phức tạp, mà vì mỗi tier bảo vệ một thuộc tính riêng: Tier-1 bảo vệ tốc độ, Tier-2 bảo vệ failover, Tier-3 bảo vệ lưu dài hạn và DR.

8 6. Thiết kế Coordinator & Aggregator

8.1 6.1 Trình bày chi tiết thiết kế

Ý chính. Hệ thống tách Coordinator và Aggregator vì Strict và Heuristic có mức cam kết khác nhau. Coordinator phục vụ Strict, chạy theo mô hình HA/Raft, có Leader tính W_{global} , quản lý ownership partition, failover/failback và fencing. Aggregator phục vụ Heuristic, dùng active-standby nhẹ hơn, chỉ gom W_h thành W_{global_h} và cung cấp trạng thái khả dụng. **Bản đồ thiết kế. Coordinator Strict.**

- **Nhận heartbeat:** đọc trạng thái worker, partition id, local watermark và `offset`/status.
- **Lưu partition status:** biết partition nào active, stale, idle, failed hoặc đang reassign.
- **Tính `Wglobal`:** lấy min watermark trên partition hợp lệ và giữ watermark đơn điệu.
- **Kiểm tra fencing token:** từ chối lệnh cũ khi leader/`term` đã thay đổi.
- **Phát hiện worker lỗi:** dùng heartbeat timeout để đánh dấu worker failed.
- **Ra lệnh reassign/failback:** chuyển quyền sở hữu partition qua quy trình có `term` và command id.

Aggregator Heuristic.

- **Nhận `Wh`:** lấy watermark heuristic từ Worker theo partition.
- **Lưu trạng thái partition:** biết partition active/idle để tính min đúng với Heuristic.
- **Tính `Wglobal_h`:** lấy min trên partition active.
- **Duy trì active/standby:** Active phục vụ request, Standby giám sát để tiếp quản.
- **Không điều phối ownership:** Aggregator không phát lệnh chuyển partition.

Dữ liệu điều khiển vào.

- **Heartbeat:** worker id, partition id, local watermark, `offset`/status.
- **Watermark heuristic:** `Wh` của từng partition.
- **Raft/lock signal:** `term`, vote/state hoặc active/standby heartbeat.

Dữ liệu điều khiển ra.

- `Wglobal`: watermark toàn cục cho Strict.
- `Wglobal_h`: watermark toàn cục cho Heuristic.
- **Command điều phối:** reassign, pause, flush, resume trong Strict.
- **Fencing metadata:** `term` và command id đi kèm lệnh.

Trạng thái cần giữ.

- **Partition owner/status:** ai đang sở hữu partition và trạng thái hiện tại.

- **Global watermark:** W_{global} hoặc W_{global_h} .
- **Raft $term$:** $term$ hiện tại của Coordinator Strict.
- **Failback state:** bước failback đang chạy nếu có.
- **Aggregator lock state:** Active nào đang giữ quyền phục vụ.

Giao tiếp.

- **Worker -> Coordinator:** heartbeat Strict mỗi 1s.
- **Worker -> Coordinator:** pull state/watermark khoảng 500ms.
- **Coordinator Leader -> Follower:** replicate state và commit khi đạt đa số.
- **Worker -> Aggregator:** gửi W_h chu kỳ ngắn.
- **Standby -> Active/lock service:** giám sát heartbeat để takeover khi Active lỗi.

Đầu ra.

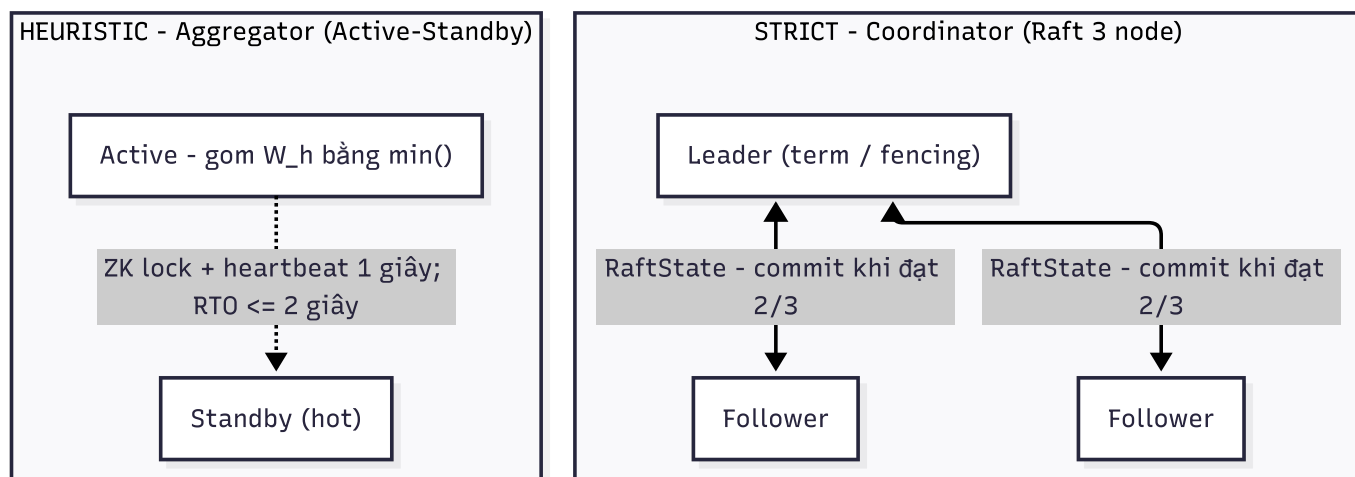
- **Cho Worker Strict:** W_{global} và lệnh điều phối partition.
- **Cho Worker Heuristic:** W_{global_h} .
- **Cho vận hành:** trạng thái owner/active/standby để quan sát control plane.

Cách chạy từng bước.

1. Trong Strict, Worker gửi heartbeat theo partition cho Coordinator Leader. Heartbeat chứa worker id, partition id, local watermark, $offset$ /status và $term$ /epoch liên quan đến quyền sở hữu.
2. Leader cập nhật bảng partition ownership, bảng watermark và trạng thái worker. Nếu phát hiện worker quá hạn heartbeat, Leader đánh dấu worker lỗi và tạo lệnh reassign partition.
3. Trước khi lệnh điều phối có hiệu lực, Leader replicate thay đổi sang Follower. Khi đạt đa số, lệnh mới được coi là committed. Term/fencing giúp Worker từ chối lệnh cũ phát ra từ Leader đã hết quyền.
4. W_{global} trong Strict được tính từ local watermark của partition, nhưng quyết định ownership/failback cũng nằm ở Coordinator. Vì vậy Coordinator không chỉ là "máy lấy min", mà là control plane có quyền điều phối.
5. Trong Heuristic, Aggregator Active nhận W_h , tính W_{global_h} và phục vụ Worker. Standby chỉ giám sát lock/heartbeat. Nếu Active mất, Standby chiếm lock và tiếp tục tổng hợp watermark, không cần replay lệnh ownership phức tạp như Coordinator.

Ví dụ vận hành.

- **Kịch bản phân tách mạng (Network Partition) trong cụm Strict:**
- Coordinator Leader hiện tại đang chạy ở Term 2, giao tiếp với Worker W1 đang gán partition P1.
- Do sự cố mạng, Coordinator Leader cũ bị cô lập hoàn toàn khỏi cụm nhưng vẫn cố gửi lệnh reassign P1 sang W2 với Term 2.
- Trong lúc đó, các Coordinator Follower bầu ra Leader mới với Term 3. Leader mới gán P1 sang W3 với Term 3.
- Khi Worker nhận được lệnh từ Leader cũ (Term 2) và Leader mới (Term 3), nhờ có cơ chế `term` (Fencing Term), Worker lập tức từ chối lệnh mang Term 2 thấp hơn và chỉ chấp nhận lệnh từ Leader mới mang Term 3.
- **Kịch bản mất điện trong cụm Heuristic:**
- Aggregator Active thu thập Heuristic Watermark `Wh` từ các Worker bị crash.
- Aggregator Standby lập tức phát hiện khóa (lease/lock) của Active bị mất, chiếm quyền điều hành và tiếp tục nhận các bản tin gRPC `Wh` từ Worker để tổng hợp `Wglobal_h`. Quá trình chuyển đổi chỉ mất < 2 giây. Các event trễ phát sinh trong 2 giây này sẽ đi vào DLQ và được sửa lại bằng Correction sau đó, không cần cơ chế đồng thuận Raft phức tạp.



Hình 10: Sơ đồ

Liên hệ. Control plane nhận watermark *map* từ node (5) qua kênh control (9), hợp nhất thành `Wglobal` / `Wglobal_h` rồi trả lại để engine đóng window (3/4). Riêng Coordinator còn giữ ownership và điều phối Failback (10); fencing token bảo vệ exactly-once đã cam kết ở 3.

8.2 6.2 Phản biện: Vì sao thiết kế như vậy

Mục Coordinator/Aggregator phải giải thích rõ vì sao hai mode không dùng chung một control plane. Strict cần quyền điều phối có fencing; Heuristic chỉ cần tổng hợp watermark nhẹ và chấp nhận correction. **Phản đề 1: Strict chỉ cần một khóa active/passive, không cần Raft.**

- **Vì sao nghe hợp lý:** hệ thống chỉ cần một Coordinator active tại một thời điểm; khóa ZooKeeper hoặc file lock có vẻ đủ để bầu leader.
- **Lý do bác bỏ:** Coordinator Strict không chỉ tính `min(watermark)`, mà còn ra lệnh chuyển ownership partition. Nếu Leader cũ bị cô lập rồi sống lại, nó có thể phát lệnh cũ song song với Leader mới. Không có `term`/fencing, Worker không biết lệnh nào hợp lệ, dẫn tới hai Worker cùng xử lý một partition.
- **Kết luận phản biện:** Strict cần Raft/fencing vì lệnh điều phối partition phải có `term` tăng đơn điệu và phải được replicate trước khi thực thi.

Phản đề 2: Heuristic cũng nên dùng Raft cho chắc chắn.

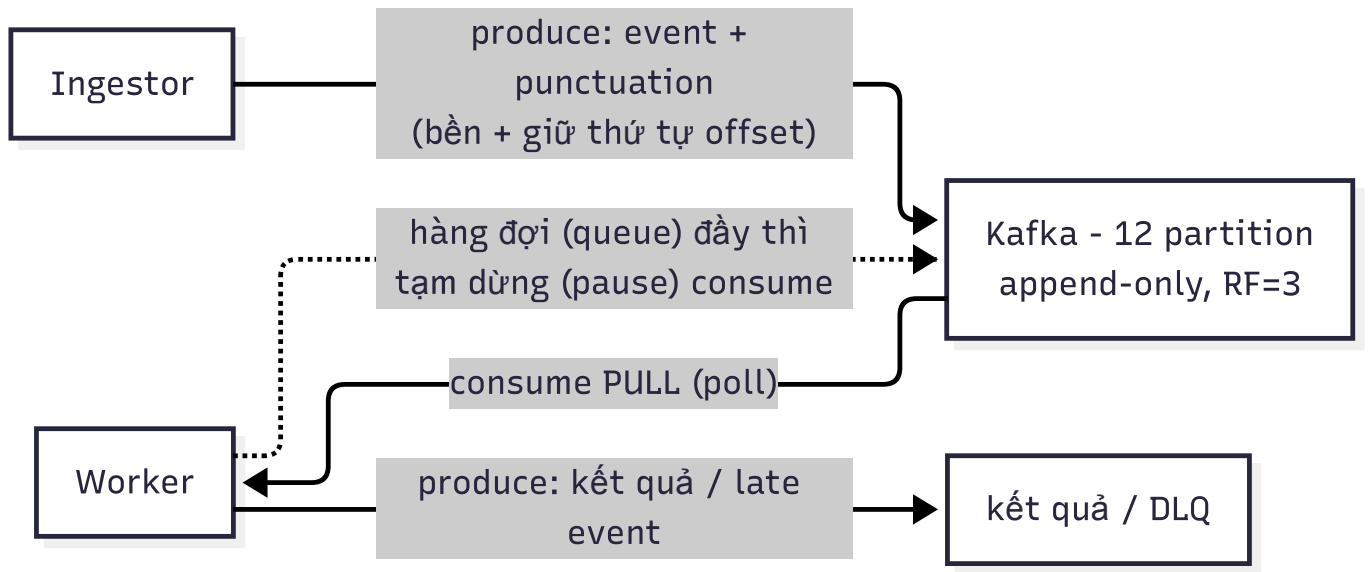
- **Vì sao nghe hợp lý:** đã có cơ chế Raft cho Strict, dùng lại cho Aggregator có vẻ tăng độ an toàn.
- **Lý do bác bỏ:** Aggregator không điều phối ownership partition và không cam kết exactly-once tức thời. Nếu Aggregator failover chậm một nhịp, tác động chính là watermark heuristic trễ hoặc late event tăng; các sai lệch này được hấp thụ bằng DLQ/correction. Raft sẽ tăng latency và độ phức tạp nhưng không bảo vệ thêm thuộc tính cốt lõi của Heuristic.
- **Kết luận phản biện:** Coordinator Strict cần nhất quán mạnh; Aggregator Heuristic chỉ cần availability nhẹ. Tách hai control plane giúp mỗi mode trả đúng chi phí cho cam kết của nó.

9 7. Giao tiếp giữa các Layer

9.1 7.1 Trình bày chi tiết thiết kế

Ý chính. Hệ thống không dùng một giao thức cho mọi thứ. Giao tiếp giữa các layer được tách theo bản chất dữ liệu: Kafka cho data plane, gRPC cho control plane, HTTP cho quan sát và thao tác vận hành, còn storage client phục vụ state/checkpoint/archive. **Bản đồ thiết kế. Kafka data plane.**

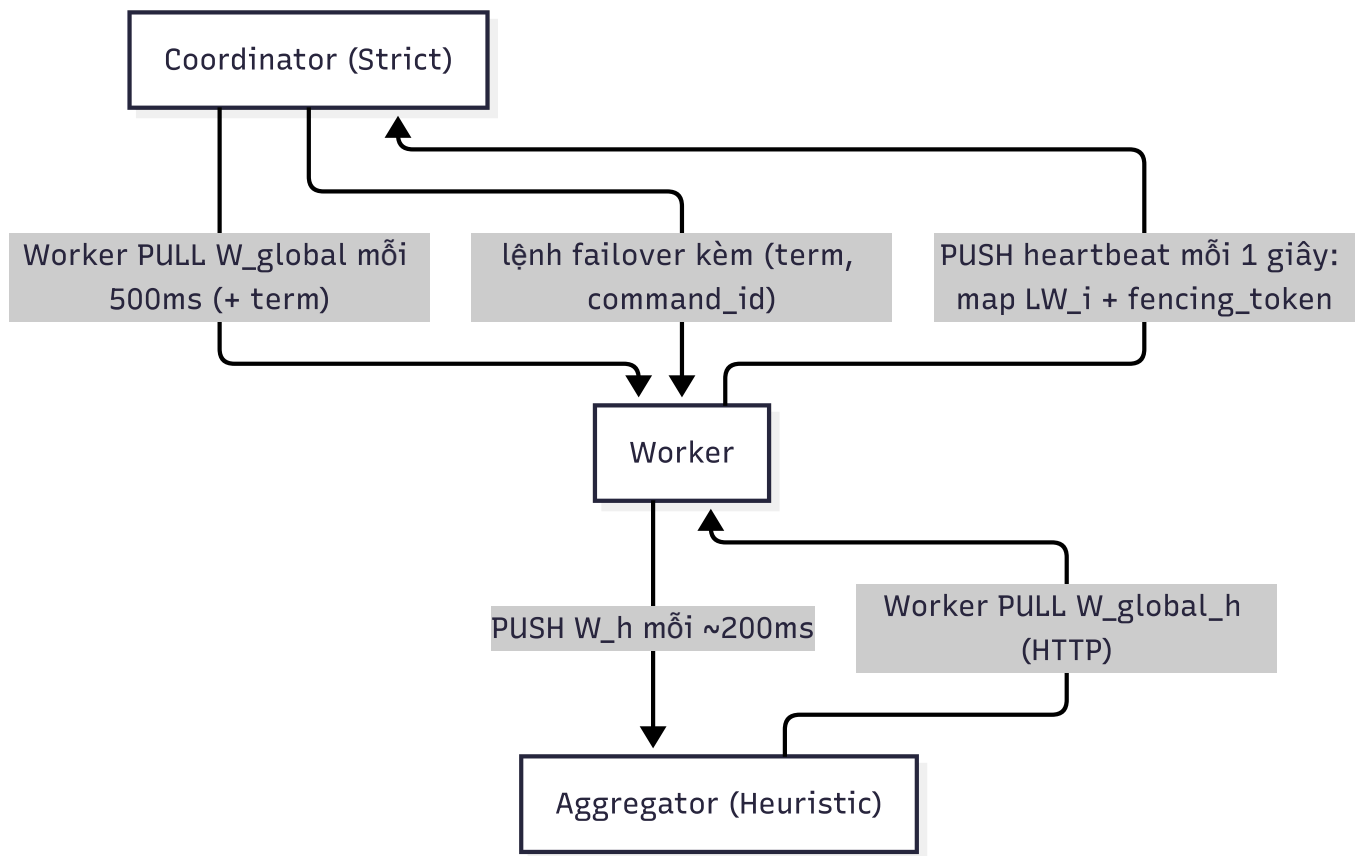
- **Payload:** event, punctuation, result, audit và DLQ.
- **Người gửi:** Ingestor hoặc Worker.
- **Người nhận:** Worker, result consumer, audit consumer hoặc DLQ/correction pipeline.
- **Yêu cầu chính:** bền, partitioned, replay được bằng `offset`.



Hình 11: Sơ đồ tầng dữ liệu (Ingestor ↔ Kafka ↔ Worker)

gRPC control plane.

- **Payload:** heartbeat, watermark, Raft vote/state, worker watermark, lệnh reassign/failback.
- **Người gửi:** Worker, Coordinator, Aggregator hoặc Raft peer.
- **Người nhận:** Coordinator, Aggregator, Worker hoặc Raft peer.
- **Yêu cầu chính:** schema rõ, latency thấp, phù hợp RPC nội bộ.



Hình 12: Sơ đồ tầng điều khiển (Worker ↔ Coordinator/Aggregator)

HTTP observability/control phụ.

- **Payload:** health, state, metrics, dashboard và fallback control.
- **Người gửi:** dashboard, operator hoặc script vận hành.
- **Người nhận:** service endpoint.
- **Yêu cầu chính:** dễ quan sát, dễ gọi thủ công, không nằm trên hot path.

Storage client.

- **Payload:** state, checkpoint, archive object.
- **Người gửi:** Worker hoặc recovery process.
- **Người nhận:** RocksDB, Tier-2 checkpoint storage, MinIO.
- **Yêu cầu chính:** persistence ngoài RAM và phục hồi được sau lỗi.

Luồng độc lập.

- **Data plane:** event lớn đi Kafka.
- **Control plane:** heartbeat/watermark đi gRPC.

- **Observability:** dashboard/metrics đi HTTP.
- **State plane:** state/checkpoint/archive đi storage client.

Đầu ra.

- **Đường dữ liệu rõ:** event đi Kafka.
- **Đường điều khiển rõ:** watermark/heartbeat đi gRPC.
- **Đường quan sát rõ:** health/metrics đi HTTP.
- **Đường state rõ:** RocksDB/Tier-2/MinIO giữ state.

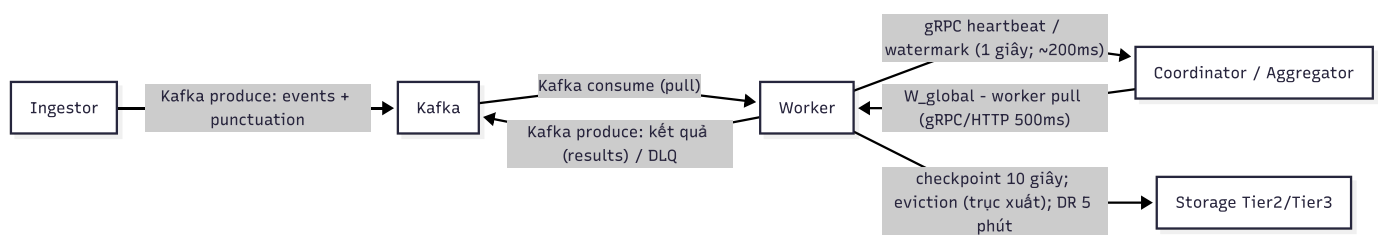
Bản đồ giao tiếp theo kênh.

Kênh	Ai gửi/nhận	Payload	Nhịp hoạt động	Khi lỗi thì sao
Kafka data plane	Ingestor -> Worker; Worker -> result/audit/DLQ	Event, punctuation, output, late event	Theo luồng dữ liệu, bền theo <i>offset</i>	Worker có thể replay từ <i>offset</i> , broker tiếp tục giữ log khi Worker pause
gRPC control plane	Worker -> Coordinator/Aggregator; Coordinator -> Worker	Heartbeat, Aggregator, Raft vote/state, lệnh reassign/failback	Chu kỳ ngắn, latency thấp	Timeout biến thành tín hiệu failover hoặc active/standby takeover
HTTP observability	Dashboard/operator -> service	Health, state, metrics, control fallback	Không nằm trên hot path	Lỗi HTTP không làm mất event; chỉ giảm khả năng quan sát/thao tác
Storage client	Worker -> RocksDB/Tier-2/MinIO	State, checkpoint, archive object	Theo event, timer hoặc window close	Retry/checkpoint/replay bảo vệ state và output

Luồng giao tiếp tổng hợp. Với Strict, event và punctuation đi qua Kafka; Worker gửi heartbeat watermark qua gRPC; Coordinator trả *W_{global}* ; Worker đóng window rồi ghi result/audit. Với Heuristic, event vẫn đi qua Kafka; Worker gửi *W_h* lên Aggregator; late event đi qua DLQ Kafka; correction cũng là một luồng dữ liệu riêng. Nhờ tách kênh, một batch event lớn không làm nghẽn heartbeat, và lỗi dashboard HTTP không làm mất dữ liệu. **Ví dụ vận hành.**

- **Tình huống quá tải dữ liệu (Data Spikes) trên Data Plane:**
- Ingestor đẩy 100,000 log/s vào Kafka topic **events**. Kafka hoạt động ổn định nhờ cơ chế lưu trữ phân vùng bền vững.

- Bộ nhớ của Worker bị quá tải do min-heap đầy. Worker lập tức kích hoạt backpressure, phát tín hiệu pause consumer Kafka cho partition đó. Kafka broker vẫn nhận log bình thường và lưu trữ an toàn, Worker tạm dừng đọc.
- Trong suốt thời gian Worker dừng consume log từ Kafka, Worker vẫn gửi heartbeat gRPC đều đặn mỗi 1s lên Coordinator thông qua kênh điều khiển (Control Plane). Nhờ đi đường out-of-band riêng, Coordinator biết Worker vẫn hoạt động bình thường, không kích hoạt cơ chế failover nhầm lẫn.
- Đồng thời, người vận hành vẫn truy cập Dashboard Web qua cổng HTTP (Observability Plane) để theo dõi các metric biểu đồ thời gian thực mà không gặp bất kỳ hiện tượng lag hay mất kết nối nào.



Hình 13: Sơ đồ

Liên hệ. Đây là mạch nối mọi mục: data plane chở event của 6, control plane chở watermark của 3/4/8, state plane chở checkpoint của 7. Việc tách kênh giải thích vì sao backlog dữ liệu (6) không làm chậm phát hiện lỗi và failover (8, 10).

9.2 7.2 Phản biện: Vì sao thiết kế như vậy

Mục giao tiếp giữa các layer phải chỉ rõ nguyên tắc chọn kênh: không chọn theo thói quen, mà chọn theo yêu cầu của payload: bền, nhanh, quan sát được hay phục vụ state. **Phản đề 1: Dùng HTTP cho toàn bộ hệ thống để dễ debug.**

- **Vì sao nghe hợp lý:** HTTP phổ biến, dễ gọi thử, dễ xem log request/response và không cần nhiều loại client.
- **Lý do bác bỏ:** event stream cần durability, partitioning và replay bằng *offset*; HTTP request trực tiếp không cung cấp các thuộc tính này. Heartbeat/watermark lại cần schema chặt và latency thấp; dùng HTTP đồng bộ cho mọi thứ dễ làm control plane bị nghẽn cùng data plane.
- **Kết luận phản biện:** HTTP chỉ phù hợp health/state/metrics/dashboard, không phù hợp thay Kafka data plane hoặc gRPC control plane.

Phản đề 2: Đưa heartbeat/watermark chung vào Kafka cho thống nhất.

- **Vì sao nghe hợp lý:** Kafka đã bền và có topic, đưa mọi message vào Kafka giúp một đường truyền duy nhất.

- **Lý do bác bỏ:** heartbeat và watermark là tín hiệu điều khiển tần suất cao, cần phản hồi nhanh để phát hiện failover. Nếu control message nằm chung cơ chế backlog với event, lúc Kafka backlog lớn thì control plane cũng chậm theo, làm phát hiện lỗi và cập nhật watermark bị trễ.
- **Kết luận phản biện:** data plane đi Kafka; control plane đi gRPC out-of-band để không bị dữ liệu lớn làm nghẽn.

Phản đề 3: Coordinator nên push W_{global} đến Worker thay vì Worker pull.

- **Vì sao nghe hợp lý:** push có vẻ realtime hơn; khi watermark đổi, Coordinator gửi ngay cho Worker.
- **Lý do bác bỏ:** push buộc Coordinator quản lý trạng thái kết nối tới mọi Worker và xử lý lại khi Leader đổi. Nếu Worker mất kết nối tạm thời, logic retry/ordering phức tạp hơn. Pull giúp Worker tự nhịp, Leader đổi thì Worker chỉ cần gọi endpoint hiện tại để lấy state mới.
- **Kết luận phản biện:** Worker pull W_{global} đơn giản hơn cho HA và đủ tốt với chu kỳ ngắn như 500ms.

Phản đề 4: Ingestor gửi heartbeat trực tiếp bằng gRPC vào Coordinator.

- **Vì sao nghe hợp lý:** Coordinator nhận tín hiệu trực tiếp, không cần thêm topic/luồng trung gian.
- **Lý do bác bỏ:** nhiều Ingestor fan-in vào một endpoint Coordinator có thể tạo điểm nghẽn. Khi Coordinator đổi Leader, toàn bộ Ingestor phải cập nhật kết nối. Đưa heartbeat Ingestor qua luồng Kafka riêng giúp hấp thụ burst và giữ lại lịch sử tín hiệu.
- **Kết luận phản biện:** Ingestor-heartbeat qua Kafka giúp giảm fan-in trực tiếp và làm control path ổn định hơn.

10 8. Thiết kế Failback 5 bước

10.1 8.1 Trình bày chi tiết thiết kế

Ý chính. Failback là quy trình đưa partition từ worker đang gánh hộ về worker cũ đã phục hồi. Worker cũ không được tự mở lại partition. Coordinator điều phối bàn giao qua 5 bước tuần tự: PAUSE -> FLUSH_ACK -> KAFKA_REASSIGN -> SEEK_RESUME -> COMPLETE. Mỗi partition đi theo máy trạng thái ASSIGNED -> REASSIGNING -> PAUSED -> ASSIGNED. **Bản đồ thiết kế. Thành phần và trách nhiệm.**

- **Coordinator Leader:** điều phối toàn bộ failback và ghi tiến độ vào control plane.
- **Worker đang gánh hộ:** đang sở hữu tạm partition sau failover, cần pause và flush.

- **Worker hồi phục:** worker cũ muốn nhận lại partition, chỉ được resume sau khi được cấp owner.
- **Kafka consumer:** cần pause/seek/resume đúng `offset`.
- **Checkpoint storage:** giữ state và `offset` an toàn trước khi chuyển owner.
- **Failback state machine:** lưu step hiện tại để tiếp tục nếu leader đổi giữa chừng.

Dữ liệu điều phối vào.

- **Partition id:** partition cần trả về.
- **From-worker:** worker đang gánh hộ.
- **To-worker:** worker hồi phục sẽ nhận lại partition.
- **Offset cuối:** `offset` an toàn sau khi flush.
- **Ack từng bước:** xác nhận từ worker để Coordinator chuyển step.

Dữ liệu điều phối ra.

- **Lệnh PAUSE:** yêu cầu worker gánh hộ dừng consume.
- **Lệnh KAFKA_REASSIGN:** đổi owner partition.
- **Lệnh SEEK_RESUME:** yêu cầu worker hồi phục đọc checkpoint và seek Kafka.
- **Term/command id:** metadata chống lệnh cũ và lệnh trùng.

Trạng thái cần giữ.

- **Failback step:** PAUSE, FLUSH_ACK, KAFKA_REASSIGN, SEEK_RESUME, COMPLETE.
- **Partition state:** ASSIGNED, REASSIGNING, PAUSED, rồi quay lại ASSIGNED.
- **Pending reassignment:** partition đang chuyển từ worker nào sang worker nào.
- **Persisted failback state:** bản ghi bền để leader mới tiếp tục đúng step.

Giao tiếp.

- **Coordinator -> worker gánh hộ:** gửi PAUSE và yêu cầu flush.
- **Worker gánh hộ -> Coordinator:** trả FLUSH_ACK kèm checkpoint id và `offset` cuối.
- **Coordinator -> worker hồi phục:** gửi SEEK_RESUME.
- **Worker hồi phục -> Coordinator:** ack đã đọc checkpoint và resume đúng `offset`.

- **Coordinator -> control plane/Raft log:** ghi step mới sau mỗi ack hợp lệ.

Đầu ra.

- **Owner cuối:** partition trở về worker hồi phục.
- **Offset continuity:** worker hồi phục đọc tiếp từ $offset + 1$.
- **Không duplicate owner:** không có thời điểm hai worker cùng consume partition.
- **Failback state sạch:** state tạm được xóa khi COMPLETE.

Thiết kế failover trước khi failback. Mục tiêu.

- **Phục hồi nhanh khi Worker chết:** partition của Worker lỗi phải có owner mới.
- **Không có hai owner cùng lúc:** partition orphaned chỉ được gán cho một Worker sống.
- **Không mất $offset$ /state:** Worker nhận thay phải đọc checkpoint và seek Kafka đúng vị trí.

Tín hiệu phát hiện lỗi.

- **Heartbeat timeout:** Worker không gửi heartbeat quá ngưỡng.
- **Partition stale:** partition không có watermark/ $offset$ mới trong thời gian cho phép.
- **Consumer lag bất thường:** lag tăng nhưng Worker không phản hồi.
- **Control command timeout:** lệnh pause/flush/reassign không được ack.

Dữ liệu failover cần có.

- **Failed worker id:** Worker bị đánh dấu lỗi.
- **Orphaned partitions:** danh sách partition đang thuộc Worker lỗi.
- **Last safe checkpoint:** checkpoint mới nhất đọc được từ Tier-2.
- **Last safe $offset$:** $offset$ đi kèm checkpoint.
- **Target worker:** Worker sống được chọn để gánh hộ.
- **Owner epoch/ $term$:** metadata fencing cho owner mới.

Luồng failover.

1. Coordinator phát hiện Worker lỗi qua heartbeat timeout.
2. Coordinator đánh dấu Worker là FAILED.

3. Coordinator lấy danh sách partition của Worker lỗi và đánh dấu chúng là orphaned.
4. Coordinator chọn Worker sống để nhận từng partition orphaned.
5. Worker mới đọc checkpoint Tier-2 của partition được giao.
6. Worker mới mở engine/state và seek Kafka từ `last_safe_offset + 1`.
7. Coordinator cập nhật owner/epoch mới và ghi trạng thái partition là **ASSIGNED**.
8. Worker mới bắt đầu consume, xử lý tiếp phần log sau checkpoint.

Trạng thái sau failover.

- **Owner tạm thời:** Worker gánh hộ trở thành owner hợp lệ.
- **Checkpoint được dùng:** hệ thống biết checkpoint nào đã dùng để phục hồi.
- **Offset tiếp tục:** Kafka consumer bắt đầu từ `offset` an toàn.
- **Failback candidate:** Worker cũ nếu hồi phục sẽ không tự lấy lại partition, mà phải đi qua failback 5 bước.

Quan hệ giữa failover và failback.

- **Failover:** xảy ra khi Worker chết, mục tiêu là giữ hệ thống tiếp tục chạy.
- **Failback:** xảy ra sau khi Worker cũ hồi phục, mục tiêu là trả partition về owner mong muốn.
- **Điểm nối:** failback chỉ bắt đầu sau khi failover đã tạo owner tạm thời hợp lệ và checkpoint/`offset` của owner tạm thời đã rõ.

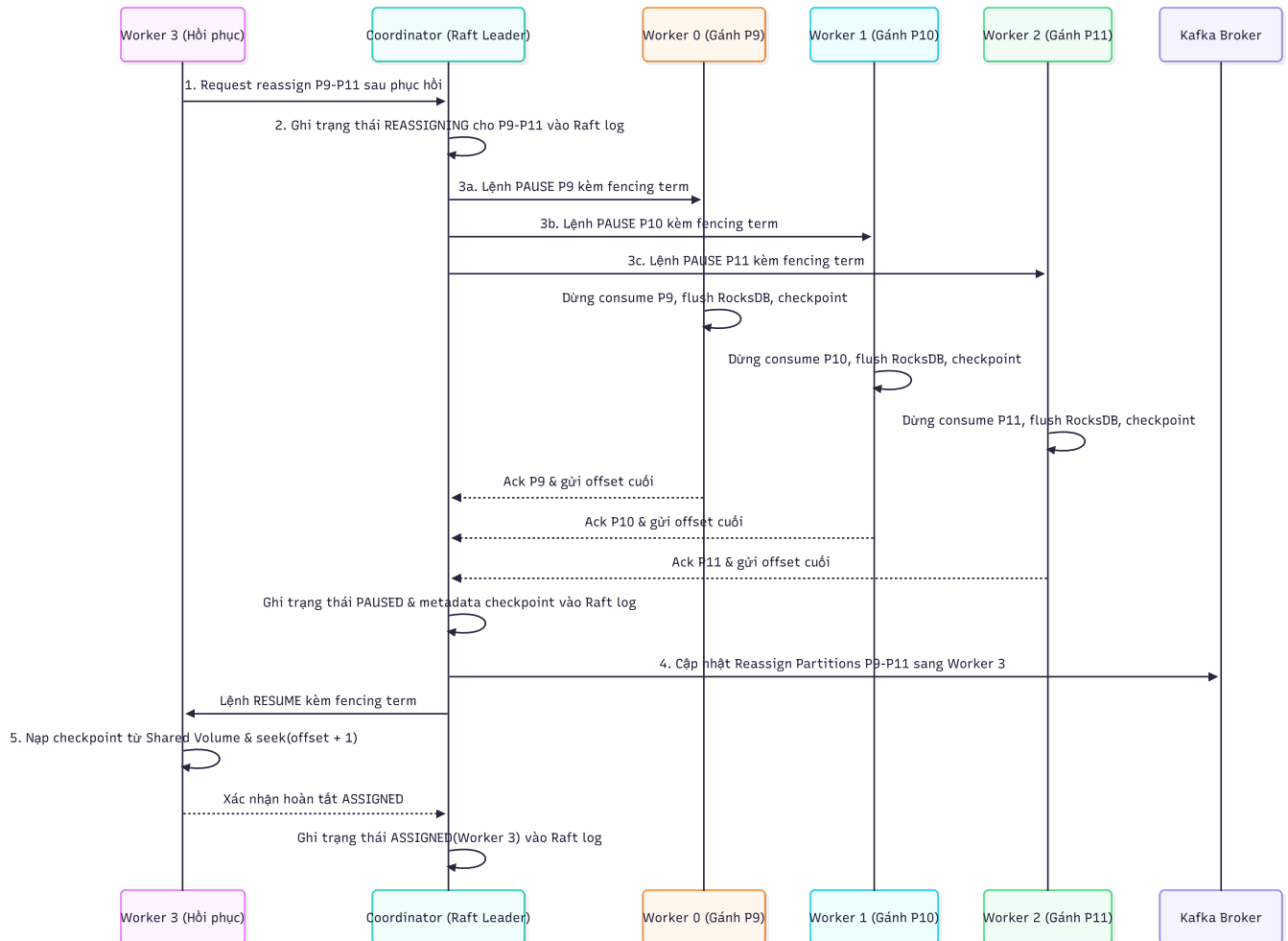
Bản đồ 5 bước failback.

Bước	Ai thực hiện	Nội dung	Điều kiện để sang bước tiếp theo
PAUSE	Coordinator gửi lệnh cho worker đang gánh hộ	Dừng consume partition đó, không nhận thêm event mới vào engine	Worker ack rằng partition đã pause và không còn poll thêm
FLUSH_ACK	Worker đang gánh hộ	Flush RocksDB/state, ghi checkpoint Tier-2, chốt <i>offset</i> cuối an toàn	Ack trả về checkpoint id và <i>offset</i> cuối
KAFKA_REASSIGN	Coordinator	Đổi owner partition từ worker gánh hộ sang worker hồi phục, tăng epoch/fencing <i>term</i>	Thay đổi owner được ghi bên trong control plane
SEEK_RESUME	Worker hồi phục	Đọc checkpoint, mở lại engine/state, seek Kafka tới <i>offset + 1</i>	Worker ack đã sẵn sàng consume từ đúng <i>offset</i>
COMPLETE	Coordinator	Đánh dấu partition trở lại ASSIGNED, xóa failback state tạm	Không còn pending command cho partition đó

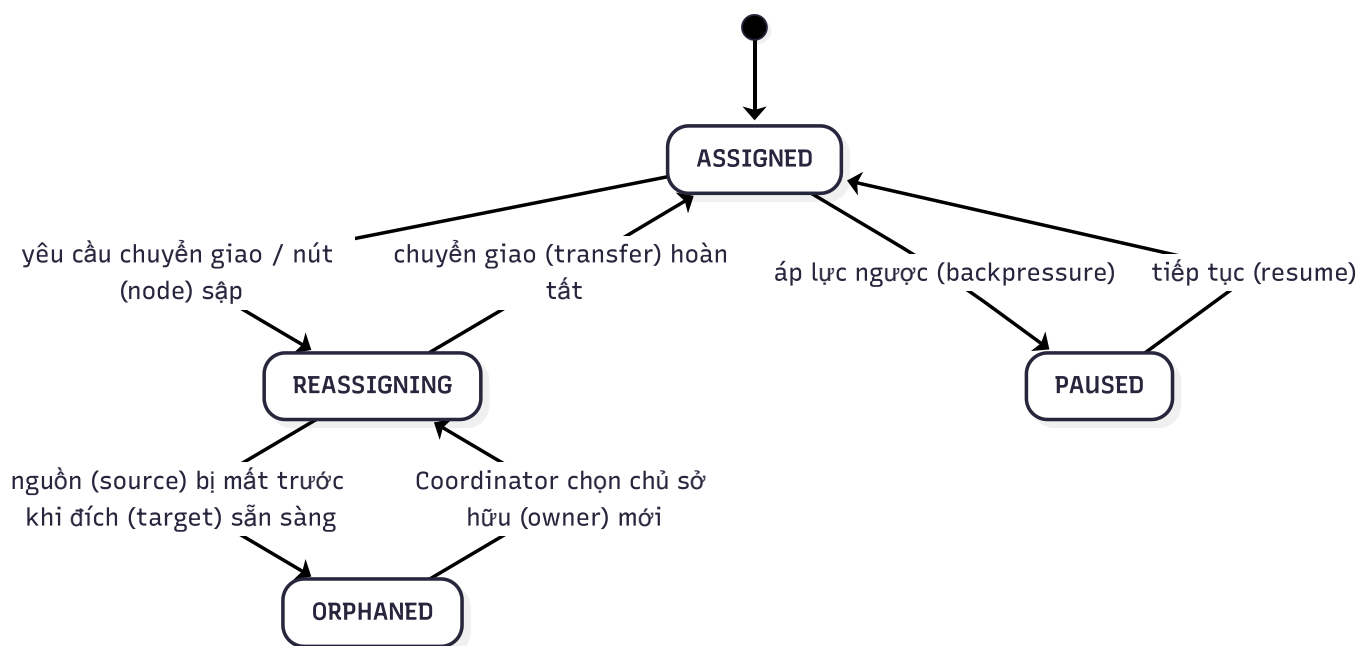
Khi failback bị ngắt giữa chừng. Failback state được persist theo partition, gồm step hiện tại, source worker, target worker, *offset* /checkpoint và *command_id*. Nếu Leader đổi ở bước FLUSH_ACK, Leader mới đọc state và tiếp tục từ FLUSH_ACK, không quay lại từ đầu. Nếu cùng một lệnh được gửi lại, Worker nhìn *command_id* để bỏ qua lệnh trùng. Nếu worker hồi phục lại chết trong lúc SEEK_RESUME, Coordinator có thể giữ partition ở worker gánh hộ hoặc mở failover mới tùy trạng thái committed cuối cùng. **Ví dụ vận hành.**

- Quy trình đưa partition P4 từ Worker gánh hộ W2 trở về Worker cũ W1 đã phục hồi:
- **Bước 1 (PAUSE):** Coordinator gửi lệnh pause partition P4 kèm *term* = 5 và *command_id* = 101 cho W2. W2 dừng consume Kafka cho P4 và ack lại Coordinator.
- **Bước 2 (FLUSH_ACK):** W2 thực hiện flush toàn bộ open window và *offset* của P4 xuống Tier-2 checkpoint, chốt *offset* an toàn cuối cùng là 950. W2 gửi FLUSH_ACK kèm *offset* 950 về Coordinator.
- **Bước 3 (KAFKA_REASSIGN):** Coordinator cập nhật bảng sở hữu partition trong control plane, gán P4 cho W1 và tăng *term* lên 6. Trạng thái P4 chuyển sang PAUSED.
- **Bước 4 (SEEK_RESUME):** Coordinator gửi lệnh resume P4 kèm *term* 6 cho W1. W1 đọc checkpoint của P4 từ Tier-2 để khôi phục state, seek consumer Kafka partition P4 đến *offset* 951, sau đó bắt đầu đọc dữ liệu và ack lại Coordinator.

- **Bước 5 (COMPLETE):** Coordinator nhận ack từ W1, chuyển trạng thái P4 thành ASSIGNED và hoàn tất quy trình failback. Suốt quá trình này, W1 và W2 không bao giờ cùng đọc partition P4 tại bất kỳ thời điểm nào.



Hình 14: Sơ đồ trình tự 5 bước



Hình 15: Máy trạng thái sở hữu partition

Liên hệ. Failback đứng trên ba thứ đã dựng trước đó: ownership và fencing token ở Coordinator (8), checkpoint Tier-2 + `offset` ở Tiered Storage (7), và đơn vị partition độc lập ở 5. Mục tiêu cuối là giữ exactly-once mà 3 cam kết.

10.2 8.2 Phản biện: Vì sao thiết kế như vậy

Mục Failback tập trung giải thích vì sao partition không được tự quay lại worker cũ và vì sao owner mới phải đọc tiếp từ `offset`/state an toàn. **Phản đề 1: Worker hồi phục tự mở lại partition cũ.**

- **Vì sao nghe hợp lý:** partition vốn thuộc Worker đó trước khi lỗi, nên khi Worker sống lại thì cho nó nhận lại ngay sẽ nhanh hơn.
- **Lý do bác bỏ:** trong lúc Worker cũ chết, partition có thể đã được Worker khác gánh hộ. Nếu Worker cũ tự consume lại, hệ thống có thể có hai owner cùng đọc Kafka và cùng ghi output cho một partition. Hậu quả là duplicate output, lệch `offset` và mất exactly-once.
- **Kết luận phản biện:** worker hồi phục không được tự nhận partition; mọi chuyển giao phải qua Coordinator và fencing `term`.

Phản đề 2: Chỉ đổi owner trong Kafka rồi cho worker mới chạy, bỏ PAUSE/FLUSH.

- **Vì sao nghe hợp lý:** giảm số bước failback, partition trở về owner cũ nhanh hơn.
- **Lý do bác bỏ:** worker đang gánh hộ có thể còn event trong buffer, state chưa flush hoặc `offset` cuối chưa ghi checkpoint. Nếu đổi owner ngay, worker mới có thể

đọc từ `offset` sai: hoặc đọc lặp quá nhiều, hoặc bỏ sót phần worker cũ đã poll nhưng chưa checkpoint.

- **Kết luận phản biện:** phải có PAUSE để dừng nhận thêm event và FLUSH_ACK để ghi state/`offset` an toàn trước khi KAFKA_REASSIGN.

Phản đề 3: Không cần ghi từng bước vào Raft log, làm tuần tự trong RAM là đủ.

- **Vì sao nghe hợp lý:** fallback là quy trình ngắn; giữ state trong RAM có vẻ đơn giản hơn.
- **Lý do bác bỏ:** Leader có thể đổi giữa chừng. Nếu state fallback chỉ nằm trong RAM của Leader cũ, Leader mới không biết partition đang ở bước nào và có thể làm lại từ đầu hoặc bỏ qua bước chưa hoàn tất. Điều này tạo duplicate command hoặc chuyển owner khi `offset` chưa an toàn.
- **Kết luận phản biện:** từng bước fallback phải được persist/replicate; `term` và `command_id` giúp từ chối lệnh cũ và bỏ qua lệnh trùng.